

*Ponajprije se zahvaljujem dragom Bogu što me vodi na životnom putu te na svemu što mi je podario i omogućio. Nadalje, želim se zahvaliti svojoj obitelji na svoj darovanoj potpori, podršci i ljubavi tijekom cijelog života i studiranja. Hvala i mojim prijateljima koji su me motivirali, hrabрили i bili uz mene tijekom napornih dana. Posebno zahvaljujem svom mentoru prof. dr. sc. Hrvoju Mlinariću na praćenju ovog rada i pomoći tijekom njegove izrade. Na samom kraju, ovaj rad i svoju diplomu posvećujem svom prerano preminulom ocu, u znak ljubavi, zahvalnosti i sjećanja.*

# Sažetak

## Planiranje obroka i praćenje zaliha namirnica na Android mobilnim uređajima

Ivan Samardžić

Ovaj rad opisuje aplikaciju koja je razvijena kao diplomski rad za upravljanje prehranom i organizaciju obroka korisnika. Zadaća aplikacije je unaprijediti organizaciju prehrane i potaknuti zdravije životne navike korisnika omogućavanjem evidencije namirnica te pametnijom organizacijom obroka. Korisnik pomoću aplikacije može samostalno planirati obroke za naredne dane na temelju dostupnih namirnica, izrađivati popise za kupovinu i upravljati svojim namirnicama. U radu je provedena i usporedba prijedloga recepata za obroke dobivenih primjenom algoritma strojnog učenja i prijedloga temeljenih na heurističkom pristupu. Također su opisani implementacija aplikacije, način prijenosa korisničkih podataka na različite uređaje i provedeni jedinični i funkcijski testovi.

**Ključne riječi:** Android, planiranje obroka, zalihe namirnica, strojno učenje, popis za trgovinu

# **Abstract**

## **Meal planning and grocery inventory tracking on Android mobile devices**

Ivan Samardžić

This master's thesis describes an application developed for nutrition management and meal planning. The purpose of the application is to improve meal planning and encourage healthier eating habits by enabling users to keep track of food items and organize their meals more efficiently. Using the application, users can independently plan meals for upcoming days based on the available ingredients, create shopping lists, and manage their food inventory. The thesis also presents a comparison of meal recipe recommendations obtained using a machine learning algorithm and recommendations based on a heuristic approach. Furthermore, it describes the implementation of the application, the method used for synchronizing user data across multiple devices, and the conducted unit and functional tests.

**Keywords:** Android, meal planning, food supplies, machine learning, shopping list

# Sadržaj

|                                             |           |
|---------------------------------------------|-----------|
| <b>Sažetak</b>                              | <b>1</b>  |
| <b>Abstract</b>                             | <b>2</b>  |
| <b>1. Uvod</b>                              | <b>5</b>  |
| 1.1. Slične aplikacije                      | 5         |
| 1.2. Opis aplikacije                        | 6         |
| <b>2. Specifikacija zahtjeva</b>            | <b>8</b>  |
| 2.1. Funkcionalni i nefunkcionalni zahtjevi | 8         |
| <b>3. Arhitektura</b>                       | <b>10</b> |
| 3.1. Arhitektura aplikacije                 | 10        |
| 3.2. Baza podataka                          | 10        |
| 3.3. Struktura koda                         | 16        |
| <b>4. Funkcionalnosti aplikacije</b>        | <b>21</b> |
| 4.1. Početni zaslon                         | 21        |
| 4.2. Prijava i registracija korisnika       | 22        |
| 4.3. Profil korisnika                       | 23        |
| 4.4. Zalihe namirnica                       | 23        |
| 4.5. Popis za trgovinu                      | 26        |
| 4.6. Upravljanje receptima                  | 28        |
| 4.7. Planiranje obroka                      | 31        |
| 4.8. Strojno učenje i heuristika            | 34        |
| 4.9. Jedinični i funkcionalni testovi       | 41        |

|                                                            |           |
|------------------------------------------------------------|-----------|
| <b>5. Korištene tehnologije i alati . . . . .</b>          | <b>47</b> |
| <b>6. Zaključak . . . . .</b>                              | <b>49</b> |
| <b>Literatura . . . . .</b>                                | <b>50</b> |
| <b>Izjava o korištenju umjetne inteligencije . . . . .</b> | <b>51</b> |
| <b>A: Kod . . . . .</b>                                    | <b>52</b> |

# 1. Uvod

U današnjem vremenu sve je obilježeno ubrzanim načinom života i sve većim brojem obaveza te se pokušava sve digitalizirati i olakšati te je iz toga proizašla ideja za razvojem aplikacije koja će pojedincu olakšati planiranje obroka i planiranje namirnica, bez obzira na to gdje se nalazi. Pa kao takva, ova mobilna aplikacija je stvorena kako bi korisniku bilo puno lakše saznati što od namirnica već ima kod kuće, bez da stoji direktno pred hladnjakom te kako bi na temelju toga mogao izabrati što jesti za koji obrok i što kupiti za buduće obroke. Unatoč velikom broju već dostupnih aplikacija, mnoge od njih nude samo djelomična rješenja poput vođenja popisa ili praćenja kalorijskog unosa, dok je integracija upravljanja zalihama često ograničena.

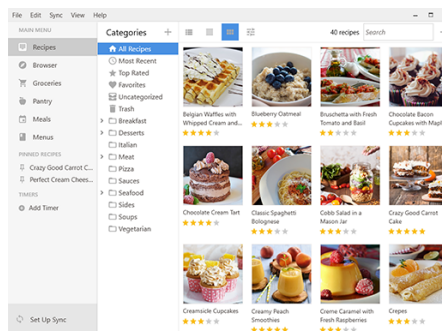
U sljedećim poglavljima biti će prikazan pregled korištenih tehnologija i koncepata korištenih u razvoju aplikacije, opis arhitekture i implementacijskih detalja, baze podataka te na samom kraju rezultati testiranja i ideje za moguća poboljšanja same aplikacije

## 1.1. Slične aplikacije

Kao što je navedeno u uvodu, postoje mnoge aplikacije koje imaju slične ciljeve i funkcionalnosti, no u svakoj postoji mjesta za napredak. U nastavku su navedene dvije najpopularnije.

### **Paprika Recipe Manager**

To je aplikacija namijenjena organizaciji recepata, planiranju obroka i popisa za kupovinu. Korisnicima omogućava spremanje recepata iz različitih internetskih izvora te planiranje obroka pomoću ugrađenog kalendara



**Slika 1.1.** Paprika Recipe Manager

## Plan To Eat

Plan To Eat je aplikacija usmjerena na jednostavno planiranje obroka i organizaciju kupovine. Korisnici mogu uvoziti vlastite recepte, raspoređivati ih u kalendar obroka te automatski generirati popise za kupovinu namirnica.



**Slika 1.2.** PlanToEat

## 1.2. Opis aplikacije

Aplikacija napravljena u sklopu ovog diplomskog rada služi za planiranje obroka i praćenje zaliha namirnica na Android mobilnim uređajima. Neki od problema koji se pokušavaju riješiti ovom aplikacijom su:

1. Upravljanje zalihama hrane - lakše će se upravljati stanjem s namirnicama u kućanstvu i na temelju toga korisnik će pri kupnji znati što mu fali ili čega ima previše. Uz to, može pratiti i količinu namirnica koje posjeduje i njihov datum isteka
2. Planiranje obroka - korisnici će moći isplanirati unaprijed što će koji dan pripremati za obrok. Na temelju odabranog recepta, korisniku će se prikazati sve nedos-

tajuće namirnice za taj obrok, čime će si olakšati planiranje kupnje

3. Izrada liste za kupnju - korisnik će moći prije odlaska u trgovinu napraviti popis potrebnih namirnica
4. Sastavljanje recepata - korisnik može upisivati svoje recepte u aplikaciju te vidjeti sve ostale recepte drugih korisnika

Ovaj proizvod je rezultat tehnološkog projekta, odnosno razvoja Android aplikacije. Sama aplikacija je usmjerena na kućanstva i osobne korisnike. Izrada aplikacije je uključivala izradu korisničkog sučelja, integraciju baze podataka, implementaciju funkcionalnosti i testiranje aplikacije pomoću jediničnih i funkcionalnih testova.

## 2. Specifikacija zahtjeva

### 2.1. Funkcionalni i nefunkcionalni zahtjevi

#### Funkcionalni zahtjevi

Funkcionalni zahtjevi su svi oni zahtjevi koji određuju što aplikacija radi, a u ovom slučaju to su:

- Registracija korisnika uz validaciju forme i spremanje u bazu podataka
- Prijava korisnika provjerom emaila i hashirane lozinke
- Održavanje sesije prijavljenog korisnika
- Mogućnost odjave i brisanja računa iz baze
- Upravljanje zalihama namirnica, što uključuje i dodavanje, brisanje, uređivanje i dohvat iz baze podataka
- Praćenje minimalne količine i upozorenja kad je količina namirnica ispod minimuma
- Praćenje roka trajanja i upozorenja za istek roka
- Upravljanje popisom za trgovinu, što uključuje dodavanje, brisanje i uređivanje količine
- Označavanje stavke popisa za trgovinu "kupljenom" uz unos roka, pri čemu dolazi do prijenosa u zalihe
- Upravljanje receptima, odnosno pregled recepata, pretraživanje po autoru i nazivu, detalji određenog recepta, te dodavanje, uređivanje i brisanje uz prethodnu pro-

vjeru autorstva

- Izračun nedostajućih sastojaka za određeni recept
- Planiranje obroka za sljedeća 3 dana po tipovima obroka - doručak, ručak i večera
- Dodavanje, brisanje i uređivanje plana obroka
- Označavanje obroka kao izvršenog uz automatsko smanjenje zaliha i zaštita od negativnih količina
- Preporuke recepata za plan uz pomoć strojnog učenja i heuristike
- Navigacija kroz glavne stavke aplikacije

## **Nefunkcionalni zahtjevi**

Nefunkcionalni zahtjevi su oni zahtjevi koji opisuju kako sustav treba raditi

- Sigurnost lozinki, što znači da se lozinke ne spremaju u čistom tekstu, nego se koristi SHA-256 hash
- Integritet podataka kroz SQL transakcije za operacije
- Ulazne validacije i obrada grešaka - obavezna polja, poruke grešaka i slične validacije
- Jedinični testovi za poslovnu logiku, UI tokove i navigaciju
- Integracijski testovi za ključne funkcionalne tokove
- Mjerena pokrivenost testovima
- Aplikacija se implementira kao Android aplikacija
- Buduće nadogradnje sustava ne smiju narušavati postojeće funkcionalnosti sustava
- Aplikacija treba biti što intuitivnija i jednostavnija kako bi bila prilagođena svim generacijama

## **3. Arhitektura**

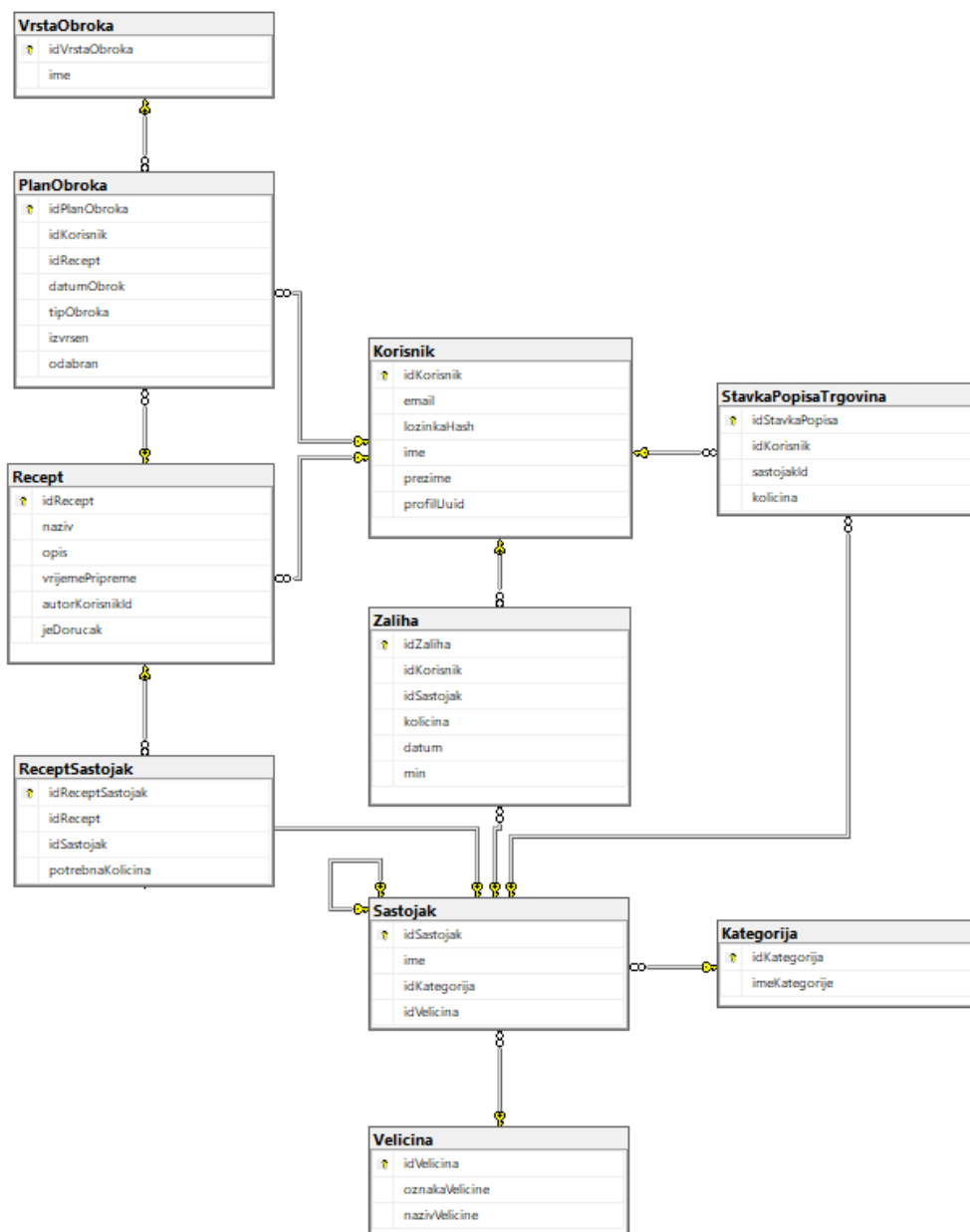
### **3.1. Arhitektura aplikacije**

Prema strukturi projekta, aplikacija je organizirana kao višeslojna klijentska arhitektura. Prezentacijski sloj čine svi dokumenti iz mape lib/(zalihe, trgovina, recepti i sl.), dok je centralna navigacija i ulazni tok definiran u main.dart. Stanje prijave, odnosno sesije, lokalno je pohranjeno pomoću SharedPreferences, mehanizma za lokalnu pohranu manjih količina podataka u obliku parova ključ–vrijednost na uređaju korisnika. Na taj se način stanje prijave zadržava između pokretanja aplikacije, čime se olakšava njezino korištenje. Što se tiče podatkovnog sloja, on je odvojen u sql\_connection.dart i dbqueries.dart. U tim datotekama se održava konekcija prema SQL Server bazi i izvršavaju sve CRUD (eng. Create, Read, Update, Delete) i transakcijske naredbe. Poslovna logika je izdvojena u posebne module, npr heuristika u heuristika.dart a ML bodovanje u ml\_scoring.dart. Sama arhitektura je dodatno prilagođena testiranju kroz apstrakcije u test\_seams.dart te kroz odvojene testove.

### **3.2. Baza podataka**

Baza podataka je važan i nužan element za ovu Android aplikaciju. U nju se spremaju i iz nje se dohvaćaju svi podatci koji se koriste u aplikaciji. U ovom projektu korištena je Microsoft SQL Server baza podataka, a alat za upravljanje tom bazom koji je korišten je SQL Server Management Studio 19. Odabrana je jer podržava sve značajke koje su potrebne ovoj aplikaciji.

## Model baze podataka



Slika 3.1. Baza podataka

Na gornjoj slici prikazan je model baze podataka. Baza se sastoji od 10 tablica, a to su VrstaObroka, PlanObroka, Recept, ReceptSastojak, Korisnik, Zaliha, Sastojak, Velicina, Kategorija i StavkaPopisaTrgovina. Svaka tablica ima svoj privatni ključ i dodatne atribute koji ju opisuju. Na slici se vide i veze između pojedinih tablica.

### Pojedinačni opis entiteta

Tablica **Korisnik** predstavlja korisnika aplikacije. Svaki korisnik ima svoj identifikator, odnosno privatni ključ te email i hashiranu lozinku za prijavu u aplikaciju, te ime i prezime.

| Naziv       | Tip podatka | Ograničenja                          |
|-------------|-------------|--------------------------------------|
| idKorisnik  | int         | Primarni ključ, povećava se sam za 1 |
| email       | varchar(50) | Not null                             |
| lozinkaHash | varchar(64) | Not null                             |
| ime         | varchar(50) | Not null                             |
| prezime     | varchar(50) | Not null                             |

Tablica 3.1. Korisnik

Tablica **Zaliha** je tablica koja označava sve zalihe u aplikaciji. Sastoji se od idZaliha koji je primarni ključ ove tablice, te je tu i idKorisnik pomoću kojeg se spaja na tablicu Korisnik i označava pripadnost zalihe određenom korisniku. Sljedeći atribut je idSastojak pomoću kojeg se spaja na tablicu Sastojak i označava koji sastojak je ta zaliha. Tu su i atributi kolicina(količina sastojka), datum(rok trajanja) i min(minimalna količina zalihe nakon koje dolazi upozorenje).

| Naziv      | Tip podatka | Ograničenja                          |
|------------|-------------|--------------------------------------|
| idZaliha   | int         | Primarni ključ, povećava se sam za 1 |
| idKorisnik | int         | Not null                             |
| idSastojak | int         | Not null                             |
| kolicina   | int         | Not null                             |
| datum      | date        | Not null                             |
| min        | int         | Not null                             |

Tablica 3.2. Zaliha

Tablica **Sastojak** označava sve sastojke koji su dostupni u bazi podataka. Sastoji se od primarnog ključa idSastojak, imena sastojka te od stranih ključeva idKategorija i idVelicina pomoću kojih se spaja na tablice Velicina i Kategorija.

| Naziv        | Tip podatka | Ograničenja                          |
|--------------|-------------|--------------------------------------|
| idSastojak   | int         | Primarni ključ, povećava se sam za 1 |
| ime          | varchar(50) | Not null                             |
| idKategorija | int         | Not null                             |
| idVelicina   | int         | Not null                             |

Tablica 3.3. Sastojak

Tablica **Velicina** je tablica koja opisuje oznaku veličine u kojima kupujemo i dobavljamo određene namirnice namirnice. To može biti komad, mililitra, kilogram i slično. Sastoji se od idVelicine koji je primarni ključ, oznakaVelicine, odnosno kom, kg, ml i slično, te od nazivVelicine koji je puni naziv oznake(komad, kilogram i slično).

| Naziv          | Tip podatka | Ograničenja                          |
|----------------|-------------|--------------------------------------|
| idVelicina     | int         | Primarni ključ, povećava se sam za 1 |
| oznakaVelicine | varchar(50) | Not null                             |
| nazivVelicine  | varchar(50) | Not null                             |

Tablica 3.4. Velicina

Tablica **Kategorija** je tablica koja opisuje kategoriju određene namirnice. Sastoji se od idKategorija koji je primarni ključ i od atributa imeKategorije što može biti meso, tjestenina, voće i slično.

| Naziv         | Tip podatka | Ograničenja                          |
|---------------|-------------|--------------------------------------|
| idKategorija  | int         | Primarni ključ, povećava se sam za 1 |
| imeKategorije | varchar(50) | Not null                             |

Tablica 3.5. Kategorija

Tablica **StavkaPopisaTrgovina** je tablica koja čuva podatke o sastojcima koji trebaju biti kupljeni u trgovini. Sastoji se od primarnog ključa idStavkaPopisa, stranog ključa idKorisnik pomoću kojeg označavamo koji korisnik je dodao namirnicu na popis, stranog ključa sastojakId pomoću kojeg određujemo koja namirnica treba biti kupljena i atributa kolicina koji određuje kolika količina te namirnice je potrebna.

| Naziv          | Tip podatka | Ograničenja                          |
|----------------|-------------|--------------------------------------|
| idStavkaPopisa | int         | Primarni ključ, povećava se sam za 1 |
| idKorisnik     | int         | Not null                             |
| sastojakId     | int         | Not null                             |
| kolicina       | int         | Not null                             |

Tablica 3.6. StavkaPopisaTrgovina

Tablica **Recept** označava recepte koji se prikazuju u aplikaciji. Sama tablica sastoji se od idRecept koji je primarni ključ, naziv koji označava naziv recepta, opis koji označava opis recepta, vrijemePripreme i autorKorisnikId koji označava koji korisnik je unio taj recept u aplikaciju. Naime, onaj koji unese ima ovlasti i za brisanje i uređivanje recepta kasnije. AutorKorisnikId je ujedno i strani ključ kojim se tablica povezuje na tablicu Korisnik. Zadnji redak označava jeDorucak koji kasnije pomaže pri predlaganju recepata pomoću heuristike i strojnog učenja.

| Naziv           | Tip podatka   | Ograničenja                          |
|-----------------|---------------|--------------------------------------|
| idRecept        | int           | Primarni ključ, povećava se sam za 1 |
| naziv           | varchar(50)   | Not null                             |
| opis            | nvarchar(MAX) | Not null                             |
| vrijemePripreme | nvarchar(50)  | Not null                             |
| autorKorisnikId | int           | Not null                             |
| jeDorucak       | bit           | Not null                             |

Tablica 3.7. Recept

Tablica **ReceptSastojak** je tablica koja prikazuje sastojke koji su potrebni za pojedini recept. Primarni ključ je tu idReceptSastojak, uz njega su tu atributi idRecept pomoću kojeg se spajamo na tablicu Recept, idSastojak pomoću kojeg se spajamo na tablicu Sastojak i potrebnaKolicina koja označava količinu određenog sastojka u nekom receptu. Svaki redak u ovoj tablici povezuje određeni recept i njegov sastojak.

| Naziv            | Tip podatka | Ograničenja                          |
|------------------|-------------|--------------------------------------|
| idReceptSastojak | int         | Primarni ključ, povećava se sam za 1 |
| idRecept         | int         | Not null                             |
| idSastojak       | int         | Not null                             |
| potrebnaKolicina | int         | Not null                             |

Tablica 3.8. ReceptSastojak

Tablica **PlanObroka** služi kako bi korisnik mogao planirati svoje buduće obroke. Sastoji se od idPlanObroka, koji je primarni ključ, onda idKorisnik pomoću kojeg se tablica povezuje na tablicu Korisnik i pomoću kojeg označavamo koji korisnik je dodao nešto u svoj plan obroka, zatim idRecept pomoću kojeg se spajamo na tablicu Recept te koji govori koji recept je odabran za neki obrok. Uz to, tu je i datumObrok, tipObroka koji se spaja na tablicu vrstaObroka i dva dodatna atributa izvršen i odabran koji kasnije služe pri predlaganju obroka.

| Naziv        | Tip podatka | Ograničenja                          |
|--------------|-------------|--------------------------------------|
| idPlanObroka | int         | Primarni ključ, povećava se sam za 1 |
| idKorisnik   | int         | Not null                             |
| idRecept     | int         | Not null                             |
| datumObrok   | date        | Not null                             |
| tipObroka    | int         | Not null                             |
| izvršen      | bit         | Not null                             |
| odabran      | bit         | Not null                             |

Tablica 3.9. PlanObroka

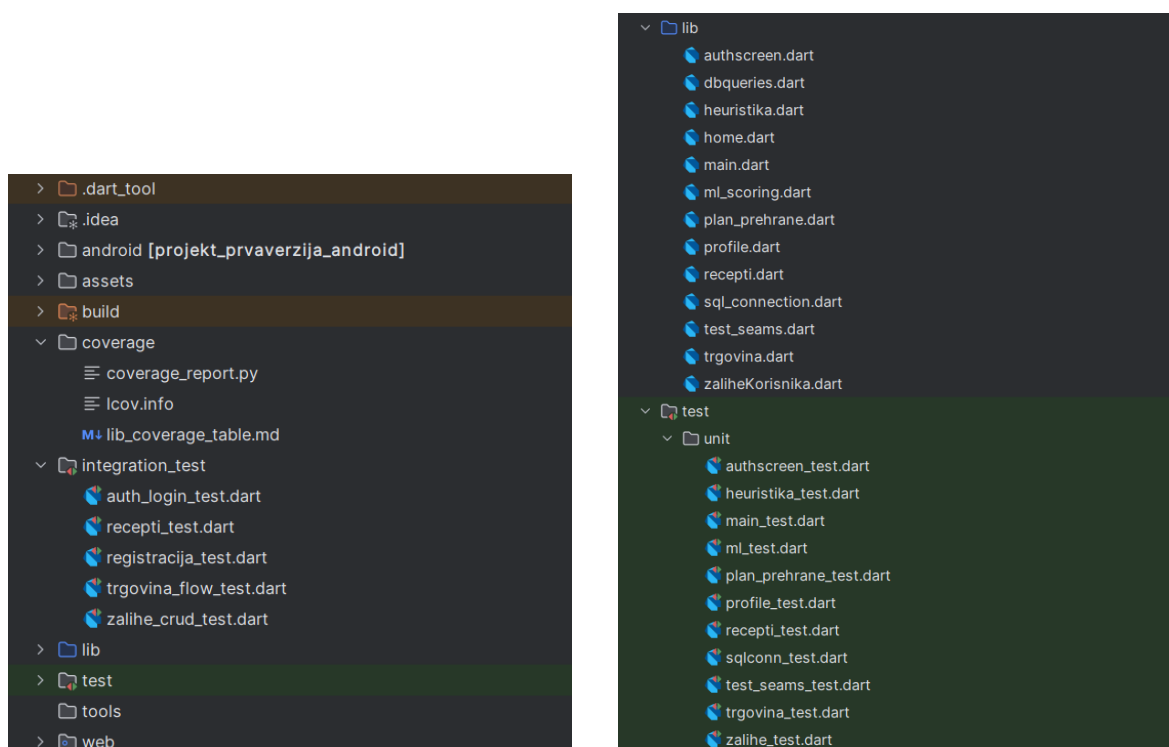
Posljednja tablica je tablica **VrstaObroka** koja služi samo za definiranje vrsta obroka. Sastoji se od idVrstaObroka koji je primarni ključ i ime koje može biti doručak, ručak ili večera.

| Naziv         | Tip podatka  | Ograničenja                          |
|---------------|--------------|--------------------------------------|
| idVrstaObroka | int          | Primarni ključ, povećava se sam za 1 |
| ime           | nvarchar(50) | Not null                             |

Tablica 3.10. VrstaObroka

### 3.3. Struktura koda

Aplikacija koristi jednostavnu strukturu koda gdje su funkcionalnosti usredotočene na direktnu komunikaciju s bazom podataka, prikaz dohvaćenih podataka u widgetima unutra Fluttera te upravljanje tim dohvaćenim podacima. Struktura projekta prikazana je na donjoj slici.



Slika 3.2. Struktura projekta

Projekt se sastoji od 4 glavne mape, a to su coverage, integration\_test, lib i test. Uz to, od važnijih datoteka tu je i pubspec.yaml. Pubspec.yaml je glavna konfiguracijska datoteka u kojoj su definirani metapodatci projekta, vanjski paketi i resursi aplikacije. U

coverage mapi se nalaze datoteke koje su korištene za provjeru pokrivenosti koda jediničnim testovima, a u test mapi su svi ti jedinični testovi koji su detaljnije objašnjeni u nastavku rada. U integration\_test mapi nalaze se funkcionalni testovi, dok je u lib datoteci "srce" aplikacije, odnosno središnji dio gdje se nalazi kod same aplikacije.

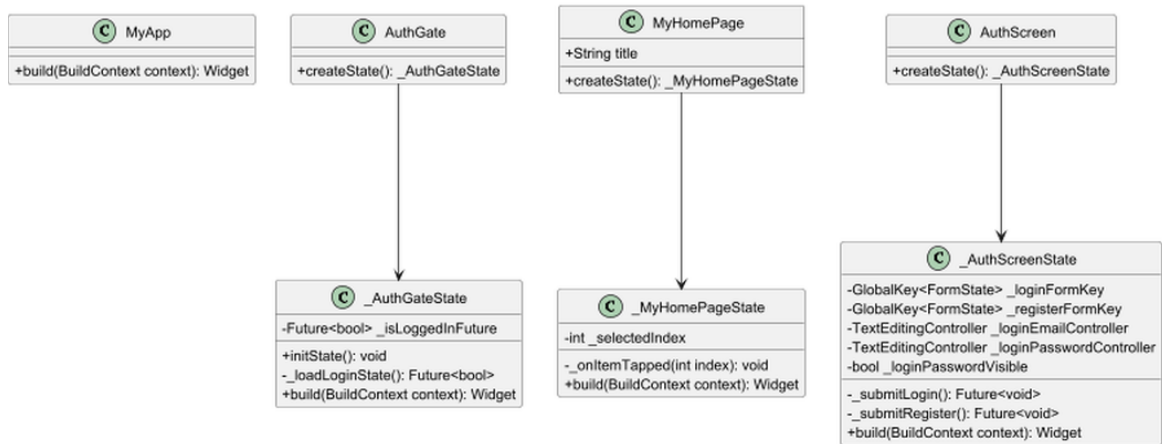
Objašnjenja lib mape:

1. authscreen.dart -> Datoteka koja implementira zaslon za prijavu i registraciju korisnika. Sadrži validaciju unosa, te obradu autentifikacije. Koristi SharedPreferences za spremanje stanja sesije.
2. dbqueries.dart -> Datoteka koja predstavlja centralni sloj za pristup bazi podataka i sadrži SQL upite za sve funkcionalnosti sustava. Implementirane su i transakcijske operacije kako bi se osigurala konzistentnost podataka kod složenijih promjena. Osim klasičnog CRUD-a, datoteka priprema i značajke za heurističko i strojno preporučivanje recepata.
3. heuristika.dart -> U ovoj datoteci definiran je heuristički model za rangiranje recepata na temelju dostupnosti namirnica, vremena pripreme i korisničkih navika. Model koristi ponderirani zbroj uz normalizaciju pojedinih signala te dodatnu penalizaciju za nisku pokrivenost zaliha. Rezultat je deterministička top-N preporuka recepata.
4. home.dart -> Datoteka sadrži jednostavnu početnu stranicu aplikacije s uvodnom porukom korisniku i pozadinskom slikom. Njezina uloga je informativna i služi kao ulazna točka nakon prijave. Ne sadrži poslovnu logiku ni interakcije s bazom podataka.
5. main.dart -> Ova datoteka definira ulaznu točku aplikacije i inicijalnu konfiguraciju. U njoj se prije pokretanja korisničkog sučelja pokušava uspostaviti SQL konekcija te provjerava stanje prijave kroz AuthGate. Također implementira glavnu navigaciju aplikacije preko donje navigacijske trake i povezivanje svih glavnih stranica.
6. ml\_scoring.dart -> Datoteka implementira izvođenje logističkog modela unutar Flutter aplikacije koristeći parametre učitane iz assets/weights.json. Učitavaju se

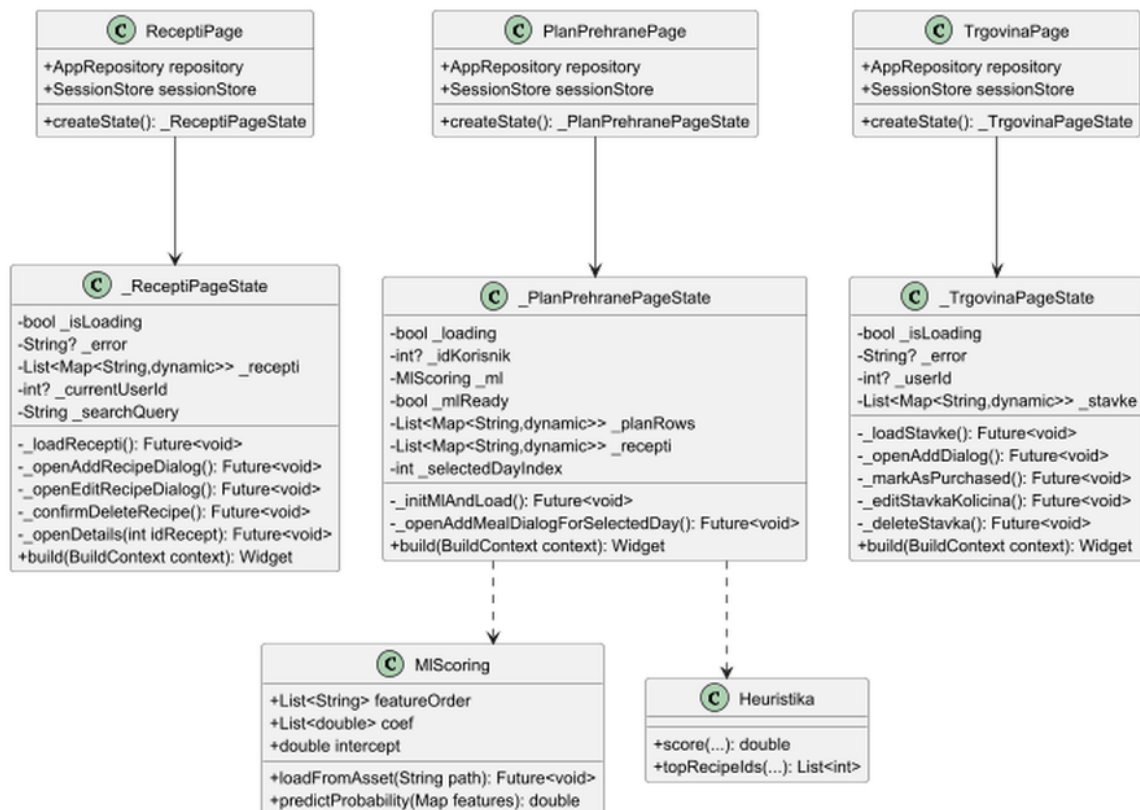
redosljed značajki, koeficijenti, presjek i parametri skaliranja, a zatim se računa vjerojatnost pomoću sigmoidne funkcije. Time aplikacija koristi prethodno istrenirani mode strojnog učenja.

7. `plan_prehrane.dart` -> Ovaj modul upravlja planiranjem obroka za trodnevni period te omogućuje dodavanje, pregled i brisanje planiranih obroka. Integrira dva pristupa preporukama(strojno učenje i heuristika) i prikazuje korisniku top 3 prijedloga za odabrani tip obroka. Također omogućuje označavanje obroka kao izvršenog, pri čemu se transakcijski ažuriraju zalihe namirnica.
8. `profile.dart` -> Datoteka implementira profil korisnika s prikazom osnovnih podataka dohvaćenih iz baze prema prijavljenom mailu. Korisniku su omogućene odjava i brisanje korisničkog računa. Nakon navedenih akcija, stanje sesije prijave se briše iz `SharedPreferences` i korisnik se vraća na početni autentikacijski tok.
9. `recepti.dart` -> Ovaj modul upravlja pregledom i pretraživanjem recepata te omogućuje dodavanje, uređivanje i brisanje recepata za autore. Uključuje i detaljan prikaz recepta sa sastojcima, vremenom pripreme i ostalim detaljima.
10. `sql_connection.dart` -> Datoteka koja služi kao sloj spajanja na SQL server kroz `sql_conn` paket. Također sadrži test seams koji omogućuju zamjenu stvarnih SQL handlera lažnim tijekom jediničnih testiranja.
11. `test_seams.dart` -> Datoteka u kojoj su definirana sučelja koja omogućuju pristup podacima i sesiji korisnika ali na način da olakšava jedinično testiranje jer omogućuje injektiranje lažnih repozitorija i kontrolu ponašanja ovisnosti.
12. `trgovina.dart` -> Datoteka implementira upravljanje popisom za trgovinu: dodavanje, brisanje i uređivanje stavki. Podržan je i tok "kupljeno" koji je dodatno objašnjen u sljedećem poglavlju.
13. `zaliheKorisnika.dart` -> Ovaj modul upravlja korisničkim zalihama namirnica kroz prikaz, dodavanje, brisanje i uređivanje stavki. Prikazuje i upozorenja za minimalne količine te rokove trajanja. Podržan je i "upsert" pristup gdje se postojeća stavka može nadopunjavati uz očuvanje ili ažuriranje postojećih podataka.

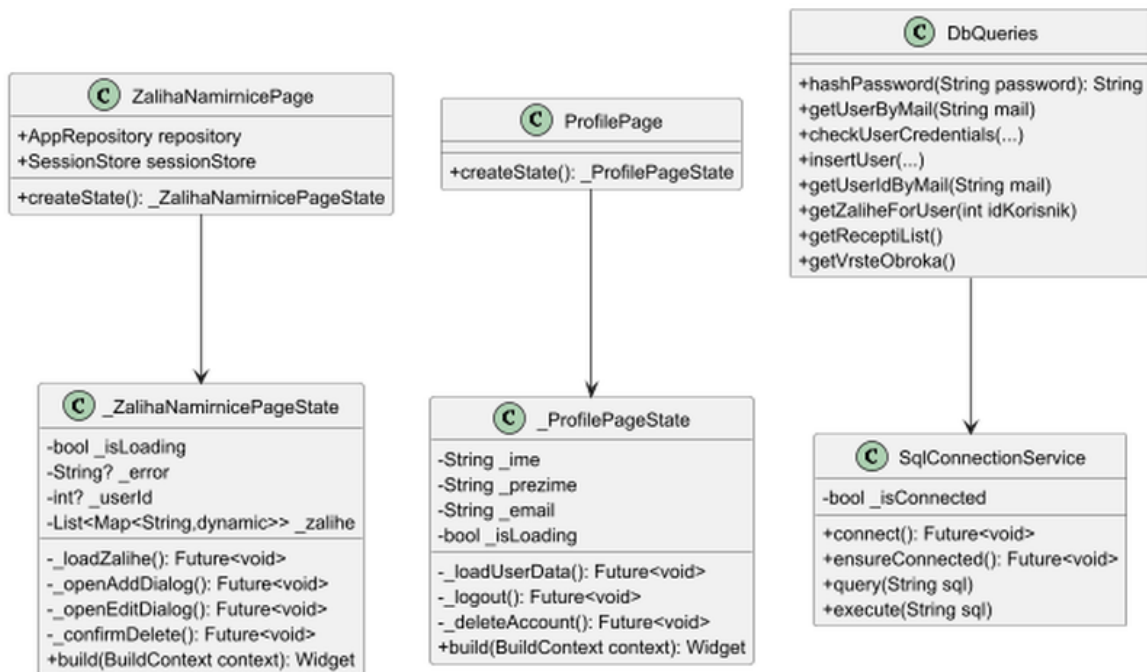
U nastavku su prikazani dijagrami klasa. Razdvojeni su na tri dijela radi bolje preglednosti.



**Slika 3.3.** Dijagram klasa osnovne strukture i autentifikacije aplikacije



**Slika 3.4.** Dijagram klasa recepata, planiranja prehrane i trgovine



**Slika 3.5.** Dijagram klasa upravljanja zalihama, korisničkim profilom i pristupom podacima

Zbog većeg broja klasa i preglednosti, dijagram klasa aplikacije podijeljen je na tri dijela. Prvi dio prikazuje klase povezane s osnovnom strukturom aplikacije i autentifikacijom korisnika. Drugi dio prikazuje klase povezane s receptima i planiranjem prehrane, uključujući komponente za heurističko i strojno učenje. Treći dio prikazuje klase za upravljanje zalihama, trgovinom i korisničkim profilom te klase zadužene za komunikaciju s bazom podataka. Radi bolje preglednosti dijagrama, veze između klase 'DbQueries' i ostalih klasa koje koriste njezine funkcionalnosti nisu prikazane. Navedene veze postoje u implementaciji aplikacije, ali su izostavljene isključivo radi smanjenja složenosti prikaza.

## 4. Funkcionalnosti aplikacije

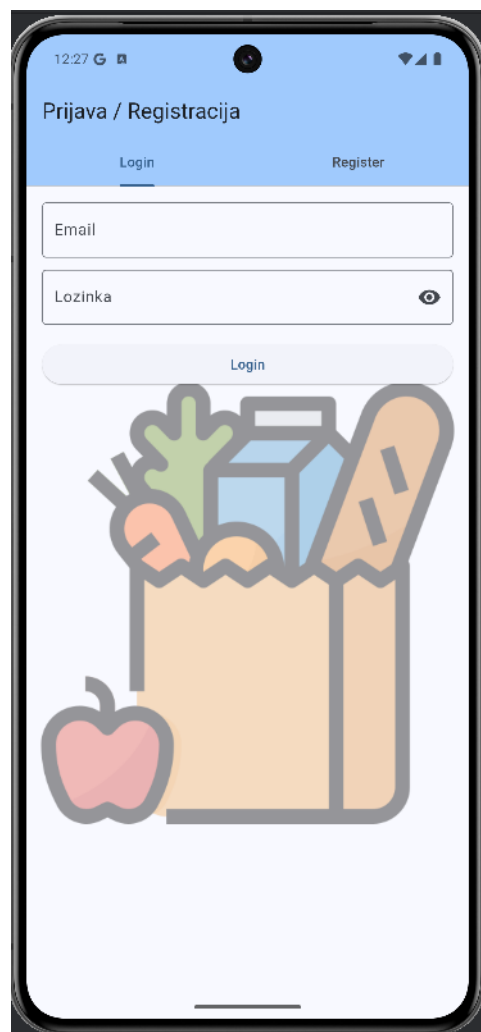
Funkcionalnosti aplikacije su napravljene tako da pokriju sve funkcionalne i nefunkcionalne zahtjeve. Cilj je bio da korištenje aplikacije zaista bude intuitivna i jednostavna svim generacijama i da ju svatko može koristiti bez posebnih uputa

### 4.1. Početni zaslon

Početni zaslon implementiran je tako da pri pokretanju prvo provjerava stanje korisničke sesije, odnosno ključ pohranjen u `SharedPreferences` `isLoggedIn`. Taj ključ vraća boolean vrijednost koja označava je li korisnik već bio prijavljen ili nije. Dok se to učitava, pokazuje se indikator učitavanja kako bi se ostvario jasan prijelaz, a ne prazan ekran. Ako je korisnik već bio prijavljen, otvara se glavni zaslon s porukom dobrodošlice, a ako nije otvara se zaslon za prijavu, odnosno registraciju korisnika. Na dnu početnog zaslona, ako je korisnik prijavljen, vidi se navigacijska traka koja služi kao vodič kroz aplikaciju, gdje korisnik može "preskakati" sa zaslona na zaslon. Time početni zaslon ima dvostruku ulogu: ulaznu kontrolu pristupa i usmjeravanje korisnika.



(a) Prijavljen korisnik



(b) Odjavljen korisnik

**Slika 4.1.** Početni zaslon aplikacije

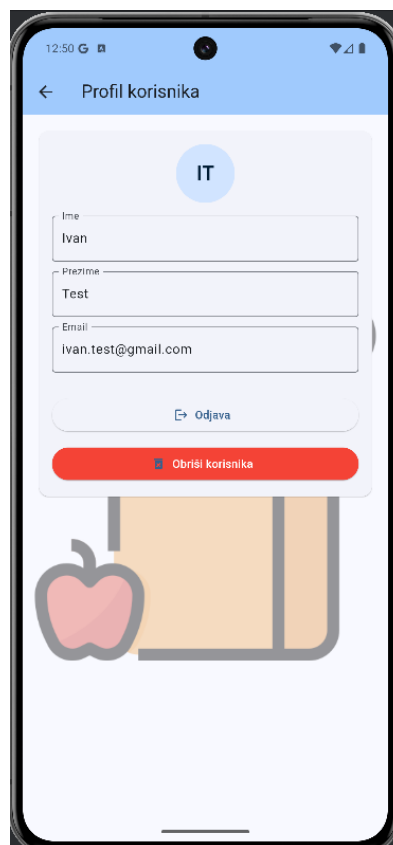
## 4.2. Prijava i registracija korisnika

Na slici gore desno prikazana je forma za prijavu, odnosno registraciju korisnika. S tim je objedinjena autentifikacija korisnika. Na vrhu se nalaze dva taba, jedan je za prijavu, drugi za registraciju. Jedina razlika je što je kod registracije potrebno dodati i ime, prezime i potvrditi lozinku. Pri unosu podataka provode se osnovne validacije (obavezno polje, format e-maila, složenost lozinke i potvrda lozinke), čime se smanjuje broj neispravnih zahtjeva prema bazi. Pri prijavi, najprije se provjerava postoji li korisnik te ako postoji ispravnost lozinke, a zatim se nakon uspješne prijave stanje sesije sprema u `SharedPreferences(isLoggedIn i loggedInEmail)` i korisnik se preusmjeri na glavni zaslon. Kod registracije se provjerava još i jedinstvenost e-mail adrese, te ako se uspješno regis-

trira, automatski je i prijavljen i nastavlja rad bez dodatnog koraka prijave.

### 4.3. Profil korisnika

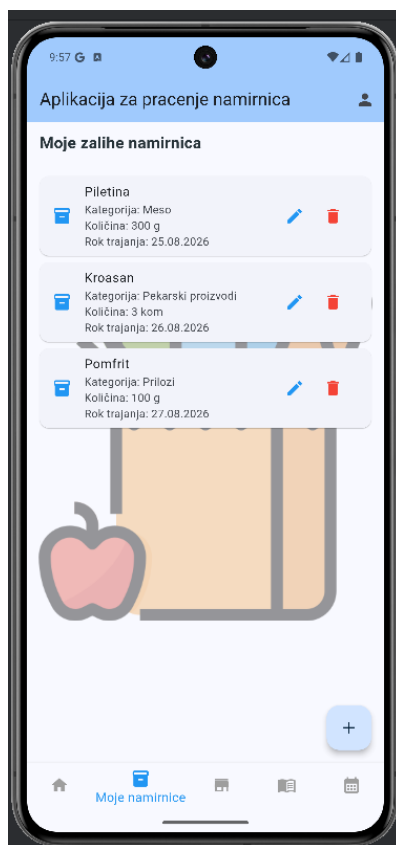
U gornjem desnom kutu početnog zaslona nalazi se mala ikona "osobe". Klikom na taj gumb otvara se profil prijavljenog korisnika. On služi za prikaz osnovnih korisničkih podataka i upravljanja računom. Podaci se dohvaćaju iz baze na temelju prijavljenog e-maila pohranjenog u lokalnoj sesiji, a korisniku su dostupne akcije odjave i brisanja računa(uz eksplicitnu potvrdu). Odjava briše podatke sesije i vraća korisnika na početni autentikacijski tok, dok brisanje računa uklanja korisnički zapis iz baze te također resetira sesiju.



**Slika 4.2.** Zaslون - profil

### 4.4. Zalihe namirnica

Na dnu početnog zaslona nalazi se ikonica "kutije" koja označava pohranu, odnosno zalihe namirnica. Klikom na taj gumb otvara se zaslon zadužen za upravljanje zaliham namirnicama.



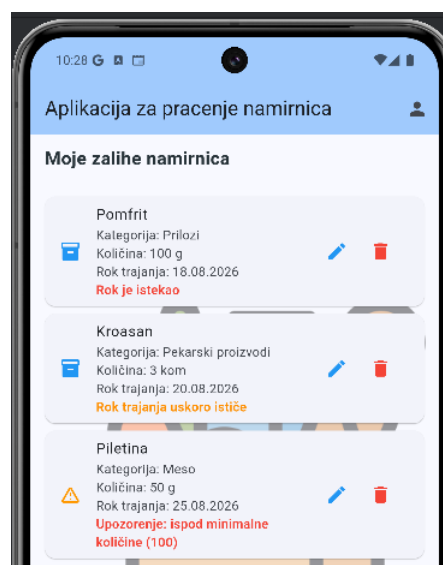
**Slika 4.3.** Zaslon - zalihe namirnica

Na slici je prikazan izgled tog zaslona. Na zaslonu korisnik vidi sve svoje trenutno dostupne namirnice, s količinom, jedinicom mjere, kategorijom i rokom trajanja. Korisnik uz svaku stavku ima i ikonu za uređivanje količine, datuma roka ili minimalne količine, te ikonu za brisanje određene namirnice, pri čemu mora dodatno eksplicitno potvrditi brisanje. Na dnu stranice nalazi se gumb za dodavanje, čime se otvara forma za dodavanje namirnice. Korisnik pri dodavanju prvo iz padajuće liste bira kategoriju, zatim odabire koju namirnicu želi dodati, te unosi količinu, minimalnu količinu (količina nakon koje mu aplikacija javlja da treba kupiti dodatnu količinu) te datum roka trajanja.

**Slika 4.4.** Forma za dodavanje namirnice

Kod dodavanja se podaci vežu na prijavljenog korisnika te se dohvaćaju kategorije i sastojci iz baze, a za već postojeću namirnicu radi se ažuriranje, odnosno uvećavanje postojeće zalihe umjesto dupliranja zapisa. Zaslone je povezan s bazom kroz CRUD operacije i osvježavanje podataka u stvarnom vremenu nakon svake promjene

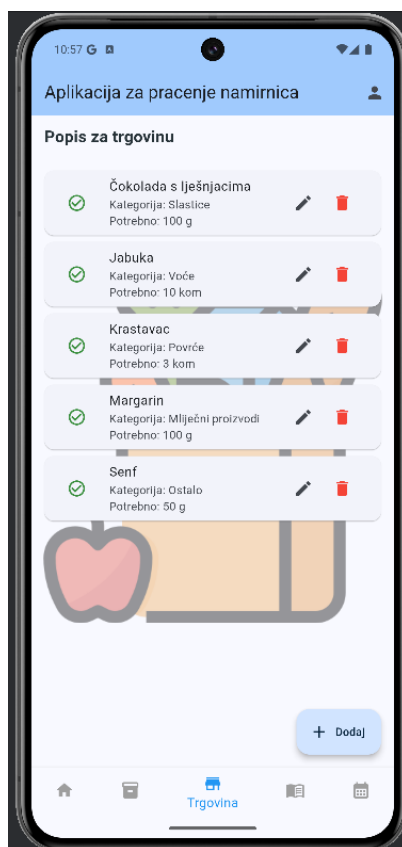
Još jedna dodatna pogodnost i funkcionalnost je to što sustav automatski označava stavke koje su ispod minimalne količine te dodatno upozorava kad je rok istekao ili uskoro istječe, čime se olakšava planiranje potrošnje i smanjuje bacanje hrane.



**Slika 4.5.** Dodatne funkcionalnosti

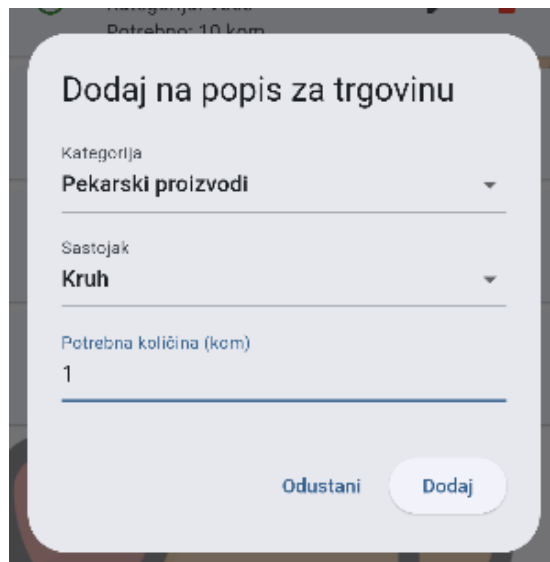
## 4.5. Popis za trgovinu

Sljedeći zaslon na redu je popis za trgovinu, a njegova ikonica je treća na navigacijskoj traci. Popis za trgovinu služi za planiranje namirnica koje korisniku nedostaju te prikazuje stavke s nazivom sastojka, kategorijom i potrebnom količinom. Svaku stavku na popisu moguće je urediti (promjena količine) ili obrisati, uz potvrdu akcije i osvježavanje liste nakon promjene.



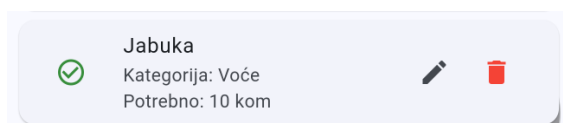
Slika 4.6. Popis za trgovinu

Na samom dnu zaslona ponovno se nalazi gumb za dodavanje namirnica. Korisnik vrlo lako može dodati novu stavku na popis za trgovinu jer se koristi obrazac s padajućom listom i time se korisnika vodi do odabira kategorije i do odabira sastojka, čime se smanjuje mogućnost pogrešnog unosa i osigurana konzistentnost naziva namirnica u aplikaciji.

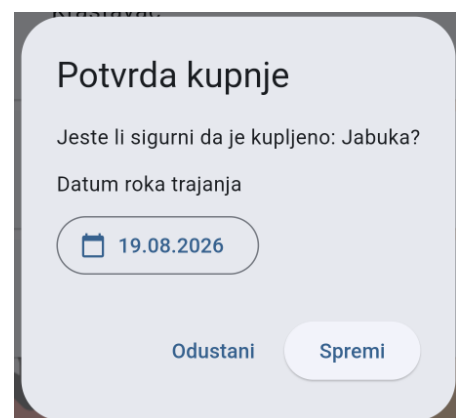


**Slika 4.7.** Forma za dodavanje namirnice na popis

Dodatna, posebna funkcionalnost je označavanje stavke kao kupljene. Kad korisnik kupi namirnicu, označi je "kupljenom" na zeleni gumb pokraj imena na popisu za trgovinu. Zatim unese rok trajanja, a aplikacija zatim premješta kupljenu količinu u tablicu zaliha i briše stavku iz popisa trgovine. Time se povezuje module popisa za trgovinu i zaliha namirnica u jedan tok, bez da korisnik mora ponovno unositi kupljene namirnice u popis za namirnice. Za to sve se koriste CRUD operacije, uz validacije (količina > 0) i time se čuva stabilnost sustava



**(a)** Element popisa sa zelenim gumbom za potvrdu kupnje

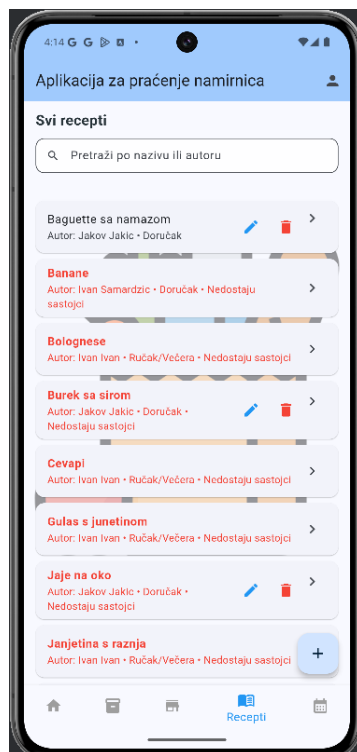


**(b)** Potvrda kupnje

**Slika 4.8.** Element popisa i potvrda kupnje

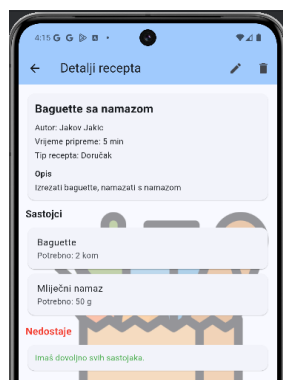
## 4.6. Upravljanje receptima

Treća ikona po redu u navigacijskoj traci vodi nas na zaslon upravljanja receptima gdje korisnik ima pregled svih recepata u sustavu.



Slika 4.9. Zaslon - recepti

Na slici je prikazan zaslon popisa recepata koji su trenutno u bazi podataka. Svaka stavka sadrži osnovne informacije o autoru, a recepti za koje korisniku nedostaju sastojci su istaknuti crvenom bojom, čime se odmah signalizira njihova trenutna neizvedivost. Ispod imena recepta je naglašeno koji je to obrok, a može biti "doručak" ili "ručak/večera". Klikom na recept, korisnik otvara detalje svakog recepta, gdje može vidjeti opis, vrijeme pripreme, tip obroka i kompletan popis sastojaka s potrebnim količinama.



**Slika 4.10.** Zaslon - detalji recepta

Kao što je vidljivo na slici 4.9., na jednoj stavci vidimo da se nalaze gumbi za brisanje i uređivanje. Oni se pojavljuju samo na receptima koje je dodao korisnik koji je trenutno prijavljen. Na slici je prikazano da je autor recepta korisnik Jakov, te se može zaključiti da je on prijavljeni korisnik. On može svoj recept brisati i uređivati, dok ostale samo može pregledati. Svaki korisnik može dodati novi recept. Pri dodavanju i kod ažuriranja koristi se forma s validacijama (obavezan naziv, vrijeme pripreme  $> 0$ , barem jedan sastojak, količina sastojka  $> 0$ ), čime je osigurana kvaliteta podataka prije spremanja. U samoj formi za dodavanje nalazi se i checkbox kojim se označava da je to recept za doručak. To se označava jer kasnije pri predlaganju recepata doručak se tretira drukčije od ručka i večere što poboljšava predlaganje recepata. Taj checkbox se i pri uređivanju može korigirati i promijeniti.

**Novi recept**

Naziv recepta

Ovo je recept za doručak ☐

Vrijeme pripreme (min)

Opis

**Sastojci recepta**

Odaberi sastojak

Potrebna količina sastojka

+ Dodaj sastojak

Nijedan sastojak još nije dodan.

Odustani Dodaj

(a) Zaslona za dodavanje recepta

**Uredi recept**

Naziv recepta  
Baguette sa namazom

Ovo je recept za doručak ☒

Vrijeme pripreme (min)  
5

Opis  
Izrezati baguette, namazati s namazom

**Sastojci recepta**

Odaberi sastojak

Potrebna količina sastojka

+ Dodaj sastojak

Baguette  
Količina: 2 kom

Mliječni namaz  
Količina: 50 g

Odustani Spremi

(b) Zaslona za uređivanje recepta

**Slika 4.11.** Dodavanje i uređivanje recepta

Jedna od bitnih dodatnih funkcionalnosti je i praćenje nedostatka neke namirnice. U detaljima recepta aplikacija dodatno računa i prikazuje sekciju "Nedostaje", gdje se za svaki sastojak koji nedostaje prikazuje koliko je dostupno, koliko je potrebno i koliko je deficit, uključujući mjernu jedinicu. To korisniku omogućuje izravnu povezanost između planiranja kuhanja i stvarnog stanja zaliha.

**Odrezak s krumpirom**

Autor: Ivan Ivan

Vrijeme pripreme: 45 min

**Opis**

Ispeci odreske na laganoj vatri, svaku stranu po par minuta. Dok se to pece, isprži krumpir. Poslužiti sa salatom.

**Sastojci**

Govedina  
Potrebno: 300 g

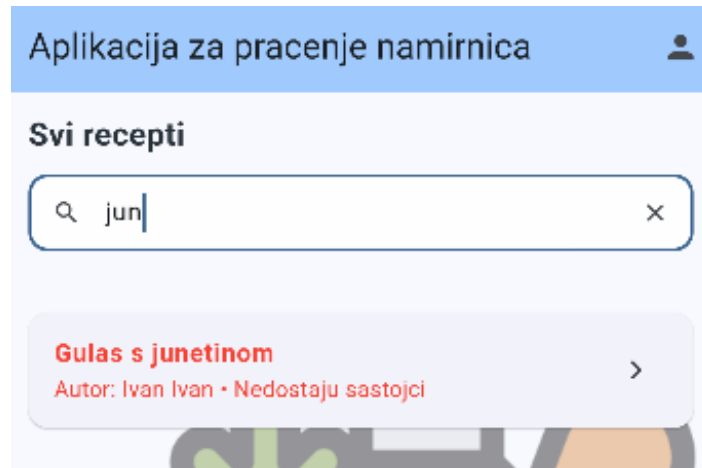
Pomfrit  
Potrebno: 300 g

**Nedostaje**

⚠ Govedina  
Nedostaje: 300 g (dostupno: 0 g, potrebno: 300 g)

**Slika 4.12.** Sastojci koji nedostaju

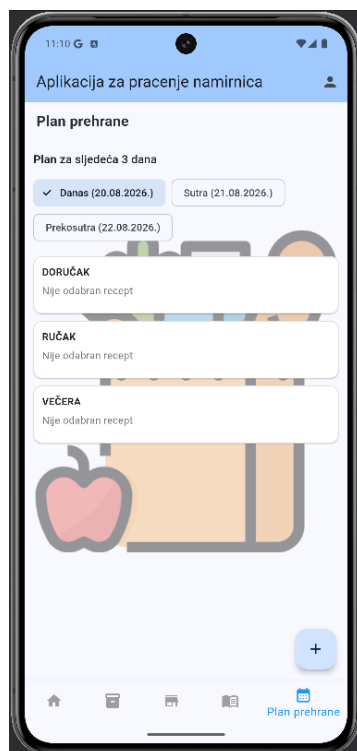
Zadnja funkcionalnost ovog zaslona je pretraživanje recepata po autoru ili nazivu recepta. Korisnik na vrhu ima traku za pretraživanje gdje unosi željeni recept ili ime korisnika čije recepte želi.



Slika 4.13. Pretraživanje recepata

## 4.7. Planiranje obroka

Posljednja ikonica u nizu na navigacijskoj traci odnosi se na zaslon za planiranje obroka. Taj zaslon omogućava korisniku planiranje obroka za 3 naredna dana, pri čemu se plan prikazuje po vrstama obroka, odnosno doručak, ručak i večera. Nakon učitavanja zaslona, aplikacija dohvaća tipove obroka, postojeći plan i listu recepata iz baze podataka. Korisnik na vrhu odabire dan(prikazan je i datum) te zatim pregledava, dodaje ili briše planirane obroke.



**Slika 4.14.** Planiranje obroka

Za dodavanje recepta, korisnik ima gumb u donjem desnom kutu. Klikom na njega otvara se forma za dodavanje recepta. Korisnik prvo odabire tip obroka koji želi isplanirati, a zatim iz padajuće liste odabire recept koji želi napraviti za taj obrok. U formi se vide prijedlozi za obroke koji su napravljeni pomoću heuristike i strojnog učenja, no o tome u sljedećem poglavlju. Korisnik može odabrati jedan od predloženih recepata, a može i normalnim putem iz baze svih recepata odabrati jedan.

(a) Forma za dodavanje obroka

(b) Dodan recept

**Slika 4.15.** Dodavanje obroka

Na slici je vidljivo da korisnik dodatno može pregledavati detalje tog recepta te ga obrisati iz plana obroka. Važan dio funkcionalnosti je označavanje obroka kao "Napravljeno". Kada korisnik potvrdi tu akciju, sustav u jednoj transakciji smanjuje zalihe prema sastojcima recepta i tada označava obrok kao izvršen. Za primjer, ako je za obrok odabran recept Piletine s pommesom i imamo dovoljno namirnica za to, nakon što kliknemo "Napravljeno", iz zalihe namirnica automatski će se smanjiti količina pilettine i pommesa za onoliko koliko je potrebno u tom receptu.

**Slika 4.16.** Obavljeni obrok

## 4.8. Strojno učenje i heuristika

U aplikaciji su implementirana dva paralelna pristupa preporuci receptata: model strojnog učenja i heuristički model. Oba pristupa koriste vrlo slične ulazne signale (vrijeme pripreme, pokrivenost zaliha, korisničke navike i prioritet roka trajanja). Prioritet roka trajanja koristi se za davanje prednosti receptima koji sadrže namirnice s bližim datumom isteka roka trajanja. Pristupi se razlikuju u načinu na koji navedene signale koriste pri donošenju odluka. Strojno učenje koristi parametre naučene iz povijesnih podataka, dok heuristika koristi ručno definirana pravila i težine. U korisničkom sučelju pri biranju recepta za obrok, oba sustava vraćaju tri najbolje rangirana prijedloga za odabrani tip obroka, čime je moguće usporediti njihovo ponašanje u istim uvjetima.

### Strojno učenje

Strojno učenje u aplikaciji realizirano je kroz nekoliko međusobno povezanih dijelova koji zajedno čine cjelinu od treniranja modela do prikaza preporuka korisniku. Sam model ne trenira se unutar Flutter aplikacije, nego se treniranje provodi zasebnoj aplikaciji napisanoj u programskom jeziku Python. Nakon treniranja, naučeni parametri modela, koji su zajednički za sve korisnike, prenose se u Flutter aplikaciju, dok se preporuke prilagođavaju pojedinom korisniku na temelju njegovih ulaznih podataka i prethodnog ponašanja.

Za treniranje nije korištena gotova aplikacija za strojno učenje, već je izrađena vlastita Python skripta `ml.py`. Za obradu tabličnih podataka u skripti se koristi biblioteka `pandas`, dok se za implementaciju postupka strojnog učenja koristi biblioteka `scikit-learn`. Sam algoritam logističke regresije nije implementiran od početka, već se koristi njegova implementacija dostupna kroz klasu `LogisticRegression`. Skripta učitava podatke iz datoteke `dataset.csv`, priprema ulazne značajke i ciljnu vrijednost te pomoću funkcije `train_test_split` dijeli podatke na skup za treniranje i skup za testiranje, pri čemu se 80% podataka koristi za treniranje, a preostalih 20% za testiranje modela. Datoteka `dataset.csv` dobivena je upitom nad bazom podataka i sadrži informacije potrebne za treniranje modela. Ulazni skup podataka sastoji se od zapisa o receptima i prethodnom ponašanju korisnika. Za treniranje se koristi šest ulaznih značajki: vrijeme pripreme recepta, pokrivenost potrebnih namirnica zalihamas korisnika, prioritet roka trajanja, broj prethodnih odabira recepta, broj prethodnih izvršenja recepta te traženi tip obroka. Ciljna

vrijednost označena je atributom `label` i poprima vrijednost 0 ili 1, ovisno o tome je li recept izvršen. Na temelju tih podataka model uči povezanost između navedenih značajki i vjerojatnosti izvršenja pojedinog recepta. Prije treniranja značajke se standardiziraju pomoću klase `StandardScaler` iz biblioteke `scikit-learn`, koja svaku značajku transformira tako da ima srednju vrijednost 0 i standardnu devijaciju 1. Time se ulazne vrijednosti dovede na usporedivu skalu i sprječava se da pojedine značajke imaju veći utjecaj na model samo zbog većeg raspona svojih vrijednosti. Nakon toga trenira se model logističke regresije pozivom metode `fit`, pri čemu model na temelju skupa za treniranje određuje koeficijente pojedinih značajki koji se kasnije koriste za izračun vjerojatnosti. Nakon treniranja model se provjerava na prethodno izdvojenom skupu za testiranje, a dobiveni parametri izvoze se za korištenje u mobilnoj aplikaciji. U datoteku `weights.json` spremaju se redoslijed značajki, koeficijenti modela, konstantni član logističke regresije (`intercept`) te vrijednosti `mean` i `scale` korištene pri standardizaciji. Na taj način priprema podataka, odabir značajki, postupak treniranja, provjera modela i prijenos dobivenih parametara u mobilnu aplikaciju definirani su u skripti `ml.py`, dok se za samu implementaciju algoritama i pomoćnih postupaka koriste funkcionalnosti biblioteka `pandas` i `scikit-learn`.

Datoteka `weights.json` predstavlja poveznicu između skripte za treniranje modela i mobilne aplikacije te ne sadrži programski kod, već podatke dobivene treniranjem modela koji su potrebni za izračun preporuka. Značajke predstavljaju ulazne podatke na temelju kojih model strojnog učenja izračunava vjerojatnost da će korisnik izvršiti određeni recept. U modelu se koristi pet značajki: `vrijemePripremeMin`, koja predstavlja vrijeme pripreme recepta u minutama, `pokrivenostZaliha`, koja označava u kojoj su mjeri potrebne namirnice dostupne u korisnikovim zalihama, `fifoSignal`, koji predstavlja prioritet potrošnje namirnica prema datumu isteka roka trajanja, `userOdabranCount`, koji predstavlja broj prethodnih odabira recepta, te `userIzvršenCount`, koji predstavlja broj prethodnih izvršenja recepta. Tijekom treniranja logističke regresije svakoj se ulaznoj značajki određuje koeficijent koji opisuje njezin utjecaj na rezultat modela. Pozitivan koeficijent povećava, a negativan smanjuje izračunatu vjerojatnost izvršenja recepta s porastom vrijednosti odgovarajuće standardizirane značajke. Osim koeficijenata, u datoteku `weights.json` spremaju se konstantni član logističke regresije (`intercept`) te srednje vrijednosti (`mean`) i vrijednosti skaliranja (`scale`) dobivene tijekom standardizacije. Spremanje tih vrijednosti potrebno je kako bi se novi ulazni podatci u mobilnoj aplikaciji standardizirali.

rali na isti način kao podatci korišteni tijekom treniranja. Model se ne trenira zasebno za svakog korisnika, već se trenira jedan zajednički model nad svim dostupnim zapisima. Personalizacija se ostvaruje prilikom izračuna preporuka jer se za pojedinog korisnika iz baze dohvaćaju njegove vrijednosti značajki, posebice broj prethodnih odabira i izvršenja pojedinog recepta. Zbog toga isti naučeni model može različitim korisnicima dodijeliti različite rezultate za isti recept. Mobilna aplikacija zatim koristi spremljene vrijednosti za izračun vjerojatnosti za nove podatke korisnika. Iz dobivenih koeficijenata vidljivo je da broj prethodnih izvršenja recepta ima najveći pozitivan utjecaj na izračunatu vjerojatnost da će korisnik izvršiti recept. Na taj se način preporuke prilagođavaju prethodnom ponašanju korisnika, pri čemu se uzimaju u obzir i ostale značajke kako preporuke ne bi ovisile isključivo o prethodno izvršenim receptima.

U mapi lib nalazi se datoteka `ml_scoring.dart` koja je zadužena za ubacivanje preporuka u našu aplikaciju. Ona prvo učitava podatke iz `weights.json` i sprema koeficijente, te zatim pomoću njih izračunava linearni rezultat logističke regresije i na njega primjenjuje sigmoidnu funkciju. Rezultat toga je broj između 0 i 1 koji predstavlja vjerojatnost da je određeni recept dobar za korisnika. U `dbqueries.dart` datoteci pomoću funkcije priprema se podatci za model na temelju stvarnog stanja baze podataka. Za svakog kandidata dohvaća vrijeme pripreme, pokrivenost zaliha, prioritet roka trajanja, koji daje prednost namirnicama s bližim rokom trajanja te broj prethodnih odabranih i izvršenih recepata za korisnika. U bazu je dodan i atribut `jeDorucak`, kojim se recepti za doručak mogu odvojiti od ostalih recepata. Time se sprječava da model predlaže recepte namijenjene za doručak za ručak ili večeru. Poseban atribut za večeru nije uveden jer se recepti namijenjeni za ručak mogu predlagati i za večeru.

Zatim u `plan_prehrane.dart` model se povezuje s korisničkim sučeljem. Pri dodavanju obroka, za odabrani tip obroka, za svakog kandidata se izračunava ML vjerojatnosti, recepti se sortiraju prema toj vrijednosti i korisniku se prikazuju 3 najbolje preporuke.

Važno je naglasiti da korisnik eksplicitno ne navodi koje namirnice preferira, već se njegove preferencije određuju kroz povijest odabira i izvršavanja recepata. Ako korisnik češće odabire i izvršava određene recepte, model strojnog učenja takve će recepte više rangirati pri budućim preporukama. Kao primjer rezultata modela strojnog učenja, na slici ispod prikazane su tri najbolje rangirane preporuke za odabranog korisnika. Preporuke se određuju na temelju trenutnog stanja zaliha i prethodnog ponašanja korisnika,

odnosno njegovih dosadašnjih odabira i izvršavanja recepata. Na prikazanom primjeru model kao najbolje rangirane recepte predlaže kruh i maslac, kruh i jogurt te burek sa sirom.



**Slika 4.17.** Primjer prijedloga recepata metodom strojnog učenja

## Heuristički pristup

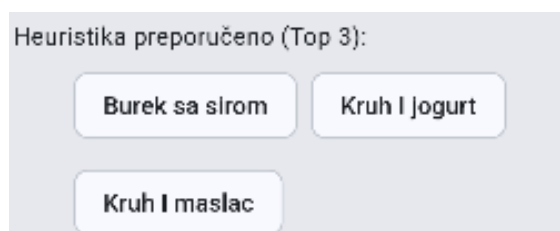
Heuristički pristup u aplikaciji koristi unaprijed definirana pravila i ručno određene težine kako bi svakom receptu dodijelio bodovni rezultat. Za razliku od strojnog učenja, heuristika ne uči težine iz podataka, nego se način bodovanja definira izravno u programskom kodu na temelju pretpostavki. Pri definiranju heurističkog pristupa pretpostavljeno je da su za kvalitetnu preporuku najvažnije korisnikove dosadašnje navike, zatim dostupnost namirnica, iskorištavanje namirnica kojima se približava rok trajanja te, u manjoj mjeri, vrijeme potrebno za pripremu recepta. Datoteka `lib/heuristika.dart` predstavlja središnji dio heurističkog sustava. U njoj je definirana klasa `HeuristikaKandidat`, koja predstavlja jedan recept zajedno sa svim signalima potrebnim za njegovo bodovanje. Za svakog kandidata čuvaju se identifikator recepta, vrijeme pripreme, pokrivenost zaliha, prioritet roka trajanja te broj prethodnih odabira i izvršenja tog recepta.

Metoda `score` u klasi `Heuristika` izračunava konačni rezultat pojedinog recepta kao kombinaciju navednih signala. Pokrivenost ima težinu 25, prioritet roka trajanja 15, a vrijeme 5. Utjecaj vremena je namjerno smanjen u odnosu na druge kriterije. Dodana je i formula koja dodatno normalizira vrijeme kojom kraći recepti dobivaju veći broj bodova, dok se njihov utjecaj postupno smanjuje kako se vrijeme povećava. Najvažniji dio heuristike predstavljaju korisnikove navike. One se određuju na temelju broja prethodnih odabira i izvršenja pojedinog recepta. Broj prethodno odabranih recepta ima manju težinu u odnosu na broj prethodno izvršenih recepata jer izvršeni recept predstavlja snažniji signal stvarne korisnikove navike. Trenutno, prethodni odabiri nose 20 bodova, dok prethodna izvršenja nose 35 bodova. Zbroj svih tih bodova kojim se ocjenjuju recepti je

100( 25+15+5+20+35).

U dbqueries.dart nalazi se metoda koja priprema podatke potrebne za heuristički algoritam. Ona iz baze dohvaća sve informacije o kandidatima (broj odabira, broj izvršenih recepata, vrijeme pripreme...). I ovdje se koristi argument jeDorucak kao i u strojnom učenju, kako bi razlikovali recepte koji su za doručak i oni koji su za ručakvečeru. Na samom kraju, kao i kod strojnog učenja, u datoteci plan\_prehrane.dart metoda povezuje rezultate iz baze s heurističkim algoritmom. Rezultati heurističkog algoritma su identifikatori 3 najbolja rangirana recepta, koji se zatim prikazuju korisniku kao heurističke preporuke. Budući da se u istom prozoru prikazuju i preporuke dobivene strojnim učenjem, moguće je izravno usporediti rezultate dvaju pristupa u istoj situaciji i nad istim skupom recepata.

Na slici je prikazan primjer rezultata heurističkog pristupa za istog korisnika kao i kod strojnog učenja. Na temelju dostupnosti namirnica, prioriteta roka trajanja, vremena pripreme te prethodnih odabira i izvršenja recepata, heuristički pristup rangira dostupne kandidate i prikazuje tri najbolje rangirana recepta. U prikazanom primjeru preporučeni su Burek sa sirom, Kruh i jogurt te Kruh i maslac.



**Slika 4.18.** Primjer prijedloga recepata heurističkim pristupom

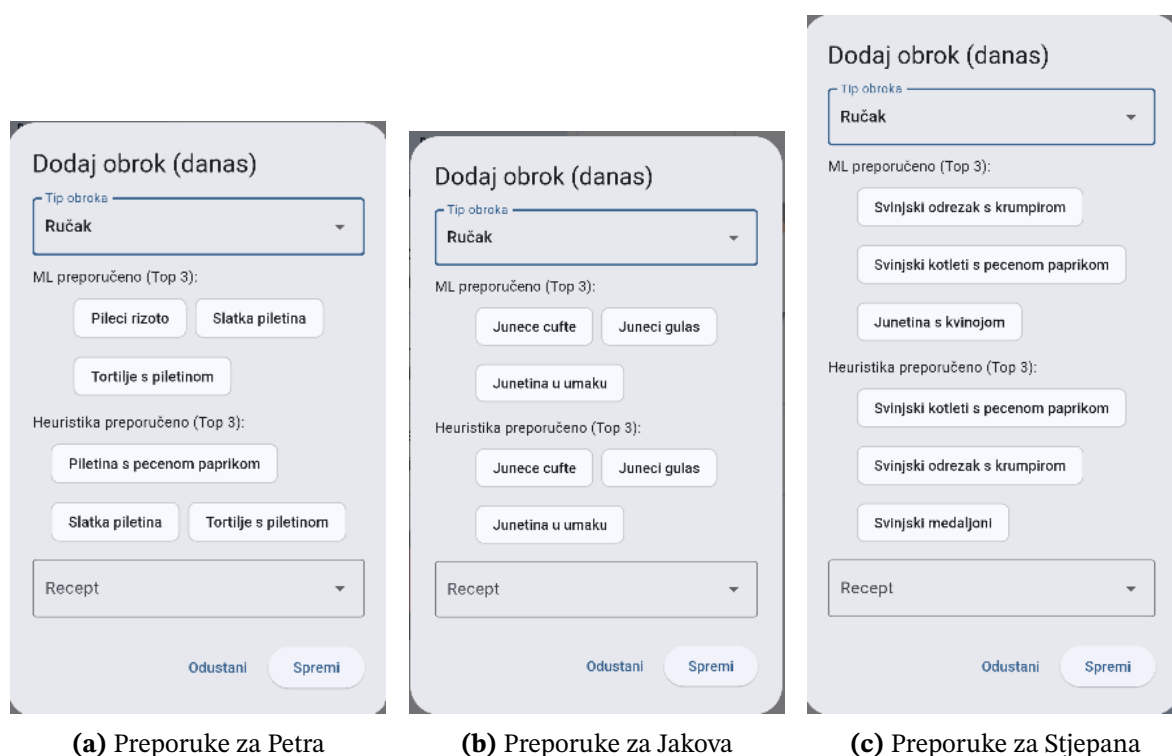
## Usporedba strojnog učenja i heuristike

Kako bi se omogućila usporedba preporuka dobivenih tim metodama, bilo je potrebno osigurati dovoljno podataka o prethodnom ponašanju korisnika. Za potrebe ispitivanja napravljena su 3 računa korisnika: Petar, Jakov i Stjepan. Svakom od njih namjerno je dodijeljena različita preferencija kod obroka. Petru je dodijeljena sklonost piletinu, Jakovu sklonost junjetini i Stjepanu sklonost svinjetini.

Za svakog korisnika generirana je povijest prehrane u trajanju od 30 dana s tri obroka dnevno, čime je dobiveno ukupno 270 zapisa. Približno 60 % odgovarajućih obroka sva-

kog korisnika sadržavalo je njegovu preferiranu vrstu mesa. Preostali obroci uključivali su i druge vrste recepata kako povijest korisnika ne bi sadržavala isključivo jednu kategoriju hrane. Na taj je način stvoren kontrolirani skup podataka u kojem je poznata dominantna preferencija svakog korisnika.

Nakon stvaranja povijesti za svakog su korisnika generirane tri najbolje rangirane preporuke modelom strojnog učenja i heurističkim pristupom. Cilj ispitivanja bio je provjeriti reagiraju li oba pristupa na različitu povijest korisnika te u kojoj se mjeri njihove konačne preporuke međusobno podudaraju. Rezultati tog testiranja su bili ovakvi:



**Slika 4.19.** Usporedba preporuka strojnog učenja i heurističkog pristupa za tri korisnika

Kod Petra su se podudarala dva od tri preporučena recepta, odnosno 66,7 % preporuka, dok se njihov poredak razlikovao. Kod Jakova je zabilježeno potpuno podudaranje, odnosno sva tri recepta preporučena su istim redoslijedom. Kod Stjepana su se, kao i kod Petra, podudarala dva od tri recepta, uz razliku u njihovu poretku. Promatrajući sva tri korisnika zajedno, dva su pristupa preporučila ukupno sedam jednakih recepata od mogućih devet, što predstavlja podudaranje od 77,8 % ako se promatra pripadnost recepta skupu tri najbolje preporuke.

Provedeno ispitivanje temelji se na usporedbi pojedinačnih preporuka dobivenih na

temelju prethodno generirane povijesti korisnika. Cilj je bio provjeriti koliko se preporuke dvaju pristupa podudaraju i koliko se prilagođavaju različitim korisničkim preferencijama. Ispitivanje nije obuhvaćalo izradu cijelih jelovnika za dulja razdoblja, pa se dobiveni rezultati odnose prvenstveno na usporedbu pojedinačnih preporuka.

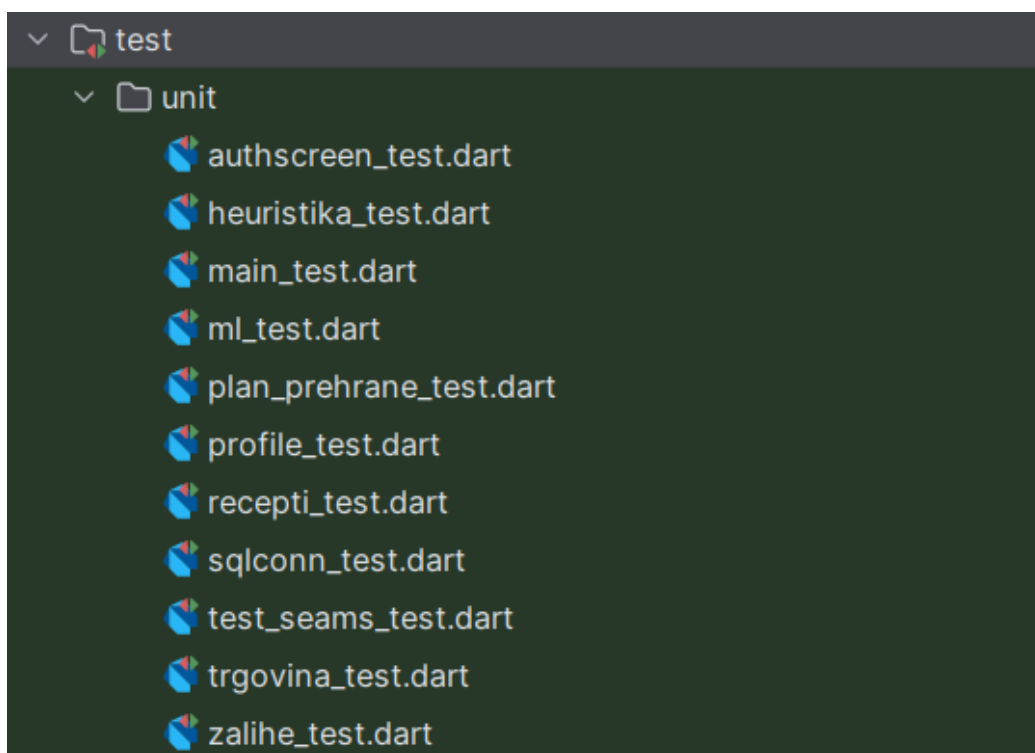
Na temelju provedene usporedbe vidljivo je da su oba pristupa proizvela personalizirane preporuke u skladu s prethodnim ponašanjem korisnika. Rezultati strojnog učenja i heurističkog pristupa u velikoj su se mjeri podudarali. Kod Petra i Stjepana podudarala su se dva od tri preporučena recepta, dok su kod Jakova sva tri recepta bila jednaka i prikazana istim slijedom. Promatrajući sva tri korisnika zajedno, sedam od ukupno devet recepata pojavilo se među tri najbolje preporuke oba pristupa, što predstavlja podudaranje od 77,8 %. Razlike između pristupa bile su izraženije u poretku pojedinih recepata nego u samom izboru preporučenih recepata. Glavna razlika između pristupa nalazi se u načinu određivanja važnosti pojedinih kriterija. Kod heurističkog pristupa težine su unaprijed ručno definirane, zbog čega je njegovo ponašanje jednostavnije objasniti i prilagoditi. Kod strojnog učenja koeficijenti se određuju treniranjem na dostupnim podacima. Zbog različitog načina određivanja važnosti pojedinih značajki dolazi i do razlika u konačnom poretku preporučenih recepata. Na korištenom skupu podataka nije uočena velika razlika između konačnih preporuka dvaju pristupa. Heuristički pristup dao je vrlo slične rezultate uz jednostavniji i transparentniji način odlučivanja. S druge strane, model strojnog učenja može se ponovno trenirati nakon prikupljanja novih korisničkih podataka, pri čemu njegova uspješnost ovisi o količini i kvaliteti dostupnih podataka za treniranje. Zbog toga bi se razlike između dvaju pristupa mogle jasnije pokazati nakon prikupljanja veće količine stvarnih korisničkih podataka. Provedeno ispitivanje temelji se na usporedbi pojedinačnih preporuka dobivenih na temelju prethodno generirane povijesti korisnika. Cilj je bio provjeriti koliko se preporuke dvaju pristupa podudaraju i koliko se prilagođavaju različitim korisničkim preferencijama. Ispitivanje nije obuhvaćalo izradu cijelih jelovnika za dulja razdoblja, pa se dobiveni rezultati odnose prvenstveno na usporedbu pojedinačnih preporuka.

## 4.9. Jedinični i funkcionalni testovi

Jedan od dodatnih zadataka u ovom diplomskom je da se prilikom pisanja koda posebna pozornost obrati na pisanje i izvršavanje jediničnih testova i na pokrivenost koda s tako napisanim jediničnim testovima te da je konačnu funkcionalnost cijele aplikacije potrebno provjeriti korištenjem funkcijskih testova.

### Jedinični testovi

Jedinični testovi provjeravaju najmanje dijelove aplikacije, poput metoda, pomoćnih funkcija i logike unutar widgeta, ali izolirano od cijelog sustava. Cilj jediničnih testova je brzo otkriti greške u implementaciji i osigurati da se promjene u kodu ne odraze negativno na već postojeću funkcionalnost. U ovom diplomskom radu jedinični testovi su usmjereni na provjeru ispravnosti poslovne logike, validacije unosa i graničnih slučajeva. U nastavku ćemo pogledati napravljene jedinične testove.



**Slika 4.20.** Jedinični testovi

Popis svih datoteka jediničnih testova

1. authscreen\_test.dart -> vezano uz authscreen.dart, testira UI tabove(Login i Regis-

ter), validacija forme, login scenarije(nepostojeći korisnik, kriva lozinka, uspješna lozinka + spremanje sesije) i register scenarije(validacija, već postojeći mail, uspješan insert + navigacija nakon uspješnog registra)

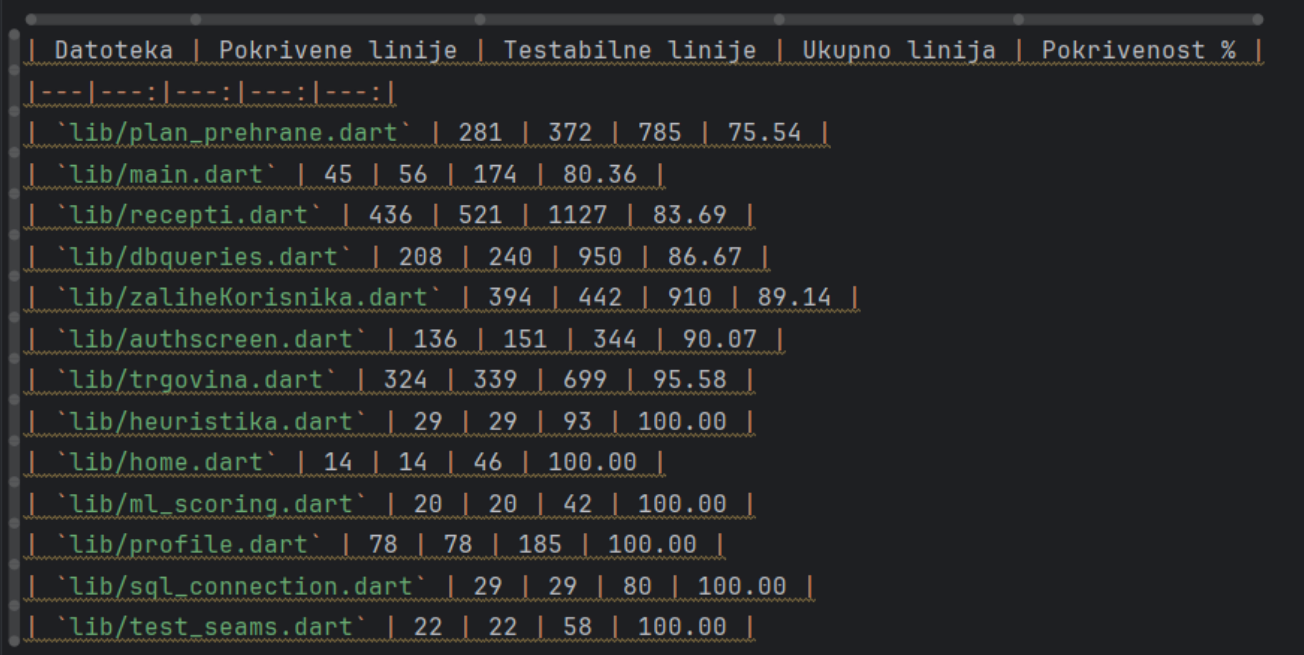
2. `heuristika_test.dart` -> vezano uz `heuristika.dart`, testira izračun score-a (granice 0-100, penalizaciju ispod praga, utjecaj vremena i user history), te top recepte(sortiranje, limit, prazan input)
3. `main_test.dart` -> vezano uz `main.dart`, testira konfiguraciju MyApp, ponašanje AuthGate za prijavljenog ili neprijavljenog korisnika i navigaciju po BottomNavigationBar karticama
4. `ml_test.dart` -> vezano uz `ml_scoring.dart`, testira učitavanje modela iz asset JSON-a, mapiranje ili parsiranje koeficijenata, te `predictProbability` grane( $z=0$ , rast signala, null/mising feature, `scale == 0` fallback)
5. `plan_prehrane_test.dart` -> vezano uz `plan_prehrane.dart`, testira render stranice s podacima plana, prazan plan za prijavljenog korisnika i slučaj bez valjanog emaila (da nema rušenja)
6. `profile_test.dart` -> vezano uz `profile.dart`, testira učitavanje profila i inicijala, fallback kada korisnik nije pronađen, izostanak DB upita bez emaila, odjavu(brisanje sesije) i brisanje računa(cancel/confirm)
7. `recepti_test.dart` -> vezano uz `recepti.dart`, testira listu recepata, pretragu, indikator nedostajućih sastojaka, error state, dodavanje recepta (validacija/uspjeh), uređivanje, brisanje i detalje recepta
8. `sqlconn_test.dart` -> vezano uz `sql_connection.dart`, testira servis konekcije, propagaciju grešaka, retry nakon greške te query putanje
9. `test_seams_test.dart` -> vezano uz `test_seams.dart`, testira da DbAppRepository pravilno prosljeđuje upite i mapira rezultate
10. `trgovina_test.dart` -> vezano uz `trgovina.dart`, testira prikaz popisa, empty/error state, označavanje kupljenog artikla, uređivanje količine, brisanje stavke i dodavanje nove stavke

11. `zalihe_test.dart` -> vezano uz `zaliheKorisnika.dart`, testira prikaz zaliha i upozorenja (istek roka ili minimalna količina), dodavanje zalihe, uređivanje i brisanje zalihe

Pokrivenost koda mjeri koliki je udio izvršivih linija koda aktiviran tijekom pokretanja testova. U ovom radu pokrivenost se mjeri nad izvornim datotekama u mapi `lib`, pri čemu se postotak računa na testabilnim linijama koda. Za svaku datoteku zasebno prikazani su broj pokrivenih i ukupnih testabilnih linija te pripadni postotak, što omogućuje precizno uočavanje slabije pokrivenih dijelova sustava. Pokrivenost koda jediničnim testovima u ovom projektu je 87.16% a pokrivenost po datotekama vidljivo na donjoj drugoj slici.

Coverage: 87.16% (2016/2313 lines)

**Slika 4.21.** Pokrivenost koda jediničnim testovima



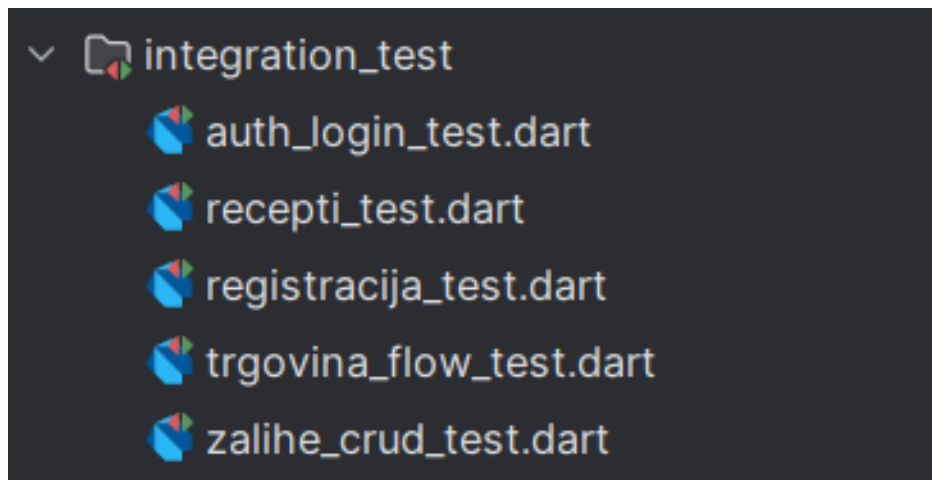
| Datoteka                                | Pokrivene linije | Testabilne linije | Ukupno linija | Pokrivenost % |
|-----------------------------------------|------------------|-------------------|---------------|---------------|
| <code>`lib/plan_prehrane.dart`</code>   | 281              | 372               | 785           | 75.54         |
| <code>`lib/main.dart`</code>            | 45               | 56                | 174           | 80.36         |
| <code>`lib/recepti.dart`</code>         | 436              | 521               | 1127          | 83.69         |
| <code>`lib/dbqueries.dart`</code>       | 208              | 240               | 950           | 86.67         |
| <code>`lib/zaliheKorisnika.dart`</code> | 394              | 442               | 910           | 89.14         |
| <code>`lib/authscreen.dart`</code>      | 136              | 151               | 344           | 90.07         |
| <code>`lib/trgovina.dart`</code>        | 324              | 339               | 699           | 95.58         |
| <code>`lib/heuristika.dart`</code>      | 29               | 29                | 93            | 100.00        |
| <code>`lib/home.dart`</code>            | 14               | 14                | 46            | 100.00        |
| <code>`lib/ml_scoring.dart`</code>      | 20               | 20                | 42            | 100.00        |
| <code>`lib/profile.dart`</code>         | 78               | 78                | 185           | 100.00        |
| <code>`lib/sql_connection.dart`</code>  | 29               | 29                | 80            | 100.00        |
| <code>`lib/test_seams.dart`</code>      | 22               | 22                | 58            | 100.00        |

**Slika 4.22.** Pokrivenost koda jediničnim testovima po datotekama

## Funkcionalni testovi

Funkcionalni testovi provjeravaju ponašanje aplikacije iz perspektive korisničkih scenarija, odnosno testiraju kako više komponenti radi zajedno u stvarnom toku korištenja. Za razliku od jediničnih testova, ovdje je naglasak na kompletnom procesu (npr. prijava korisnika, rad na zalihami itd.), a ne samo na pojedinim metodama. Time se potvrđuje

da sustav kao cjelina isporučuje očekivanu funkcionalnost. U nastavku ćemo pogledati napravljene funkcionalne testove.



**Slika 4.23.** Funkcionalni testovi

Integration\_test mapa sadrži end-to-end testove koji pokreću stvarnu aplikaciju i kliku UI kao pravi korisnik. Većina testova ovisi o stvarnim podacima u bazi i postojećem korisniku te većina njih ima cleanup korake tako da run testova bude ponovljiv i da se baza ne puni testnim podacima. Datoteke u mapi su:

1. auth\_login\_test.dart

Ovaj test testira osnovni auth tok: login + odjava + povratak na auth ekran. Scenarij prvo pokrene aplikaciju i prepozna stanje, ako je korisnik već na home ekranu, test radi odjavu prije same prijave. Za login korake on pronalazi dva polja za unos podataka, unosi hardcodirani email i lozinku i zatim klikne na Login. Nakon login-a očekuje naslov home ekrana "Aplikacija za praćenje namirnica", te na kraju radi odjavu i provjerava da je prikazan gumb Prijava/Registracija. čime vraća sustav u početno stanje

2. recepti\_test.dart

Ovaj test testira dodavanje recepata sa sastojcima i naknadno brisanje recepata. Prvo s home stranice navigiramo na stranicu s receptima, te zatim odmah po dolasku na stranicu gleda postoji li već recept kojeg želi napraviti. Ako postoji i ako je autor isti, briše ga. Ako ga nema, otvara formu za unos recepta, te unosi naziv, vrijeme pripreme i opis. Zatim dodaje sastojak iz dropdown liste, te ima mogućnost

listanja liste zbog njene dužine. Za svaki sastojak unosi količinu, te zatim odabire "Dodavanje recepta". Nakon spremanja, provjerava da je recept vidljiv na zaslonu, te kasnije briše recept kako bi doveo sustav u stanje u kojem ga je pronašao.

### 3. registracija\_test.dart

U ovom testu fokus je na registraciji i validaciji duplikata maila. U testu se prvo osigurava da se kreće iz odjavljenog stanja. Test obuhvaća 2 glavna scenarija. U prvom scenariju se generira jedinstveni email pomoću funkcije vremena kako bi se izbjegle kolizije, te se popunjavaju polja(ime, prezime, email, lozinka, te potvrda). Zatim se verificira redirect na home ekran nakon uspješne registracije. Ulazi se u profil, provjerava da je novonastali račun s tim emailom prikazan. Zatim se čiste testni podatci brisanjem računa i provjerava povratak na auth ekran. U drugom scenariju korisnik se pokušava registrirati koristeći hardcodirani, već korišteni mail, te se očekuje poruka da je taj email već registriran.

### 4. trgovina\_flow\_test.dart

Ovo je ujedno najveći test u funkcionalnim testovima. Pokriva tokove popisa za trgovinu uključujući i uređivanje i stanje "kupljeno". Prvo dolazimo na ekran za upravljanje popisom za trgovinu, te pokušava pronaći stavku koju želi dodati u testu te ju briše ako postoji. U prvom testu, dodaje određenu stavku. Prvo u formi za dodavanje iz dropdown liste odabire kategoriju, zatim sastojak i na kraju potrebnu količinu. Tu testira i verifikaciju da se ne može dodati količinu "0". Zatim ide na uređivanje iste te stavke, ulazi kroz gumb za uređivanje, mijenja količinu i zatim potvrđuje sa "Spremi". Nakon toga obavlja "kupnju". Označuje stavku kao "kupljenu", potvrđuje kupnju u dijalogu te se nakon toga očekuje da je stavka obrisana. Drugi test je dodavanje stavke istim tokom kao u prethodnom testu te se odmah testira brisanje stavke te potvrda odluke.

### 5. zalihe\_crud\_test.dart

Ovaj test pokriva CRUD tok za korisničke zalihe namirnica(dodavanje, uređivanje, brisanje).Test najprije putem navigacijske trake otvara zaslon zaliha korisnika. Ako korisnik nekim slučajem nije ulogiran, prvo se odradi logiranje korisnika. Odmah nakon toga, funkcija briše ciljnu namirnicu ako već postoji. Zatim slijedi scenarij dodavanja. Otvara se dijalog za dodavanje namirnice. Zatim se bira katego-

rija, a zatim i sam sastojak kroz dva dropdowna. Unosi se količina i minimalna količina koju korisnik želi imati, te se odabire datum roka trajanja i zatim spremanje. Nakon spremanja provjerava da je namirnica prikazana s očekivanom količinom. Nakon toga testira uređivanje, odabire gumb za to, te u formi za uređivanje mijenja količinu, spremi i nakon toga provjeri novu vrijednost. Na samom kraju obriše tu namirnicu i potvrdi da je nestala s liste. U ovom testu je važno odabrati sastojak koji postoji pod tom kategorijom i pod tim nazivom u bazi podataka.

## 5. Korištene tehnologije i alati

### Android Studio

Aplikacija je pisana u Android Studiu. To je integrirano razvojno okruženje namijenjeno za Android platformu. Vrlo moćan alat za razvoj aplikacija napravljen od strane Google-a za izradu, testiranje i deploy Android aplikacija. Također omogućava testiranje aplikacije na fizičkom uređaju i pomoću emulatora. [1]

### Flutter

Flutter je razvojni okvir otvorenog koda koji je razvio i održava Google. Razvojni programeri koriste Flutter za izradu korisničkih sučelja aplikacija za više platformi koristeći jedinstvenu bazu programskog koda. Flutter je prvenstveno bio namijenjen razvoju mobilnih aplikacija, dok danas podržava razvoj aplikacija za šest platformi: iOS, Android, web, Windows, macOS i Linux. [2]

### Dart

Dart je besplatan programski jezik otvorenog koda namijenjen razvoju kvalitetnih aplikacija za različite platforme. Jednostavan je za korištenje te pruža dosljednu i sažetu sintaksu uz strogu tipizaciju i moderne značajke. Dart također podržava značajku hot reload, koja omogućuje prikaz promjena u pokrenutoj aplikaciji gotovo odmah nakon izmjene programskog koda. [3]

### SSMS

SQL Server Management Studio (SSMS) je integrirano okruženje za upravljanje bazom podataka. SSMS pruža alate za različite zadatke, uključujući alate za konfiguraciju, nadzor i administraciju instanci SQL servera i baze podataka. Najvažniji alati su alati za

brisanje, kreiranje i ažuriranje tablica te za izvršavanje raznih SQL upita. [4]

## **GitHub**

GitHub je web platforma za verzioniranje i upravljanje kodom koja omogućuje programerima lakše snalaženje. Na GitHub-u se koristi Git, distribuirani sustav za upravljanje izvornim kodom. GitHub ima mogućnosti gledanja unazad, odnosno pogled na neke starije verzije, pa ako je potrebno lako možemo pogledati nešto što smo obrisali ali nam sada ponovno treba. [5]

## 6. Zaključak

U okviru ovog rada razvijena je Android aplikacija za upravljanje zalihama namirnica i planiranje obroka. Implementirane su glavne funkcionalnosti potrebne za svakodnevno korištenje aplikacije, među kojima su praćenje zaliha namirnica, planiranje obroka, upravljanje receptima i izrada popisa za kupovinu. Poseban naglasak stavljen je na sustav preporuke recepata, za koji su implementirana dva različita pristupa: model strojnog učenja i heuristički pristup. Model strojnog učenja temelji se na logističkoj regresiji, a preporuke prilagođava korisniku na temelju podataka poput prethodnih odabira i izvršenja recepata, dostupnosti namirnica, prioriteta roka trajanja i vremena pripreme. Heuristički pristup koristi iste informacije, ali konačni rezultat određuje pomoću unaprijed definiranih pravila i težina. Na taj su način implementirana dva različita načina rangiranja recepata koja omogućuju stvaranje personaliziranih preporuka na temelju prethodnog ponašanja korisnika. Za provjeru rada oba pristupa provedeno je testiranje na tri korisnika s različitim preferencijama. Za svakog korisnika generirana je povijest prehrane u trajanju od 30 dana, čime je ukupno dobiveno 270 zapisa. Rezultati su pokazali da oba pristupa uspješno prepoznaju dominantne korisničke preferencije te da na temelju svih parametara daju slične rezultate. Dobiveni rezultati pokazuju da na korištenim podacima strojno učenje nije ostvarilo značajno poboljšanje preporuka u odnosu na heuristički pristup. Heuristički pristup dao je vrlo slične rezultate, uz jednostavniji i transparentniji način određivanja važnosti pojedinih kriterija. S druge strane, prednost strojnog učenja nalazi se u mogućnosti automatskog određivanja koeficijenata na temelju podataka i ponovnog treniranja modela nakon prikupljanja novih podataka. Zbog toga bi se razlika između pristupa mogla jasnije pokazati nakon duljeg korištenja aplikacije i prikupljanja veće količine stvarnih korisničkih podataka. Razvijena aplikacija ispunjava postavljene ciljeve rada te pokazuje mogućnost primjene različitih pristupa preporučivanju recepata u sustavu za planiranje prehrane.

## Literatura

- [1] Oliver Baker. What is android studio? [Mrežno]. Adresa: <https://www.intelivita.co.uk/blog/android-studio>
- [2] AWS Amazon. What is flutter? [Mrežno]. Adresa: <https://aws.amazon.com/what-is/flutter/>
- [3] Dart. What is dart? [Mrežno]. Adresa: <https://dart.dev/>
- [4] Microsoft. What is sql server management studio (ssms)? [Mrežno]. Adresa: <https://learn.microsoft.com/en-us/ssms/sql-server-management-studio-ssms>
- [5] GitHub. Github. [Mrežno]. Adresa: <https://github.com/>

## Izjava o korištenju umjetne inteligencije

Tijekom izrade diplomskog rada korišten je alat ChatGPT(OpenAI) i GitHub Copilot kao pomoć pri razvoju programskog rješenja i u malom dijelu u izradi dokumentacije. Alati su najviše korišteni za analizu programskog koda te za otklanjanje raznih pogrešaka u kodu. Što se tiče korištenja u pisanju dokumentacije, korišten je za stilsko oblikovanje pojedinih dijelova.

**Osvrt na korištenje umjetne inteligencije:** Sadržaj koji je generiran korištenjem AI-ja nije korišten bez prethodne provjere. Svaki kod koji sam koristio sam prvo provjerio te se uvjerio da ga razumijem i tek nakon toga se služio njime. Što se tiče prijedloga AI-ja, često je dolazilo do netočnih prijedloga, što je ponekad bila i otežana okolnost, no takve bi primjere najčešće izbacio te odradio sam. Najviše je korišten u pomoći generiranja raznih widgeta.

**Izjava o korištenju umjetne inteligencije:** Potvrđujem da sam tijekom izrade ovog diplomskog rada koristio umjetnu inteligenciju u skladu s Politikom primjerenog korištenja umjetne inteligencije na Fakultetu elektrotehnike i računarstva, te da umjetna inteligencija nije zamijenila moje kritičko razmišljanje niti moj doprinos.

## **Privitak A: Kod**

Programski kod aplikacije priložen je u posebnom privitku, dok su u radu opisane funkcionalnosti i uloge pojedinih dijelova implementacije.